Applied Mathematics from the Notices of the AMS

Selected Articles for Scientists and Engineers

Edited and adapted from *Notices of the American Mathematical Society*
2000–2025

A Computational Mathematics Book Series

July 26, 2026

Articles reprinted with permission from *Notices of the American Mathematical Society*.

Original articles:

- Urschel, J. (2025). Numerical Stability in Gaussian Elimination. *Notices*, 72(6).
- Rojas, J.M. (2025). An Introduction to Algorithmic Fewnomial Theory Over $\mathbb{R}$. *Notices*, 72(5).
- Golub, B. (2026). Spectral Methods in Microeconomics. *Notices*, 73(6).
- Balestriero, R., Humayun, A.I., Baraniuk, R.G. (2025). On the Geometry of Deep Learning. *Notices*, 72(4).
- Lim, L.-H., Nelson, B.J. (2023). What Is... an Equivariant Neural Network? *Notices*, 70(4).
- Rieck, B. (2025). Topology Meets Machine Learning: An Introduction Using the Euler Characteristic Transform. *Notices*, 72(7).
- Chen, N., Wiggins, S., Andreou, M. (2025). Taming Uncertainty in a Complex World: The Rise of Uncertainty Quantification. *Notices*, 72(3).
- Boutin, M., Kemper, G. (2025). New Developments in Global Positioning. *Notices*, 72(8).
- Bishop, P.R., Chenoweth, S., Fleurantin, E., Ogueda-Oliva, A., Sander, E., Seay, J. (2026). 3D Printing of Invariant Manifolds in Dynamical Systems. *Notices*, 73(1).
- Bar-Lev, D., Sabary, O., Yaakobi, E. (2022). Exciting Coding Problems for DNA-based Storage Systems. *Notices*, 69(11).
- Emanuello, J.A., Ridley, A. (2022). The Mathematics of Cyber Defense. *Notices*, 69(6).
- Baez, J.C., Foley, J.D. (2021). Operads for Designing Systems of Systems. *Notices*, 68(6).
- de Souza, R.S., Ishida, E.E.O., Krone-Martins, A. (2025). A Brief History of Inference in Astronomy. *Notices*, 72(10).
- Bacher, T., Garrote-López, M., Görgen, C., Neubert, M.J. (2026). Making Mathematical Online Resources FAIR. *Notices*, 73(5).
- Foschiatti, S., Kittenberger, A., Scherzer, O. (2025). Deciphering Scrolls with Tomography. *Notices*, 72(10).
- Tao, T. (2025). Machine-Assisted Proof. *Notices*, 72(1).

ISBN: XXX-X-XXXX-XXXX-X

Contents

Preface

This book collects fifteen articles from the *Notices of the American Mathematical Society* (2000–2025) that are exceptionally valuable for applied scientists and engineers. Each article was chosen for three criteria:

1. **Computational substance**: The article presents algorithms, code, or numerical methods—not just theory.
2. **Engineering relevance**: The mathematics solves or illuminates real problems in physics, biology, data science, security, or engineering.
3. **Accessibility**: Written for a broad mathematical audience; no specialized background beyond advanced calculus/linear algebra is assumed.

The articles have been lightly edited for consistency: notation is unified across chapters, cross-references are added, and each chapter includes **Julia/Python code** demonstrating the key algorithms. A companion repository at `https://github.com/ams-notices-book/code` contains runnable notebooks.

How to use this book

- **As a reference**: Each chapter is self-contained. Dip in for the topic you need.
- **As a course text**: Chapters 1–3 (numerical linear algebra, fewnomials, spectral methods) form a "computational foundations" sequence. Chapters 4–6 (ML geometry, equivariance, topological ML) are a "geometric deep learning" module. Chapters 7–9 (UQ, GNSS, dynamical systems) are "uncertainty and dynamics." Chapters 10–12 (DNA storage, cyber defense, operads) are "modern applications." Chapters 13–15 (astronomy, FAIR data, tomography) are "data-intensive science." Chapter 15 (machine-assisted proof) looks forward.
- **As a code cookbook**: Every algorithm shown in the text has a runnable implementation in the companion repo.

— The Editors

Part I

Computational Foundations

CHAPTER 1

Numerical Stability in Gaussian Elimination

John Urschel
Communicated by *Notices* Associate Editor Emilie Purvine

> "In some sense, numerical analysis is a very old field... But with the advent of the computer age in the 1940s, which made numerical calculations on a scale that was previously unimaginable not only possible but easy, understanding the stability of these algorithms became increasingly urgent."
>
> — John Urschel

Abstract. Gaussian elimination is the oldest algorithm in linear algebra, the most popular method for solving linear systems, and the workhorse behind every 'A\' command in MATLAB, Julia, and Python. Yet its numerical stability remained mysterious for decades. This chapter traces the story from von Neumann and Goldstine's 1947 analysis through Wilkinson's breakthrough to modern understanding of growth factors, pivoting strategies, and the surprising fact that partial pivoting—theoretically unstable in worst case—works beautifully in practice.

1.1 Introduction: The Algorithm Behind Every Linear Solve

You have solved a linear system today. Whether you wrote `A \b` in Julia, `numpy.linalg.solve(A, b)` in Python, or `A\` in MATLAB, you called Gaussian elimination (via LAPACK's `getrf`/`getrs`). It is the default, the standard, the algorithm that powers finite element codes, circuit simulators, structural analysis, and countless engineering computations.

And yet: *why does it work?*

The algorithm is ancient. The earliest variant appears in Chapter 8 of *The Nine Chapters on the Mathematical Art* (Han Dynasty, ∼200 BCE), solving a 3×3 system for rice yields. Gaussian elimination is simple enough to teach to middle schoolers. But as von Neumann and Goldstine

discovered in 1947, when you run it on a computer with finite precision, *errors compound*. The question of stability—whether small rounding errors produce small changes in the answer—became "absolutely basic to numerical mathematics" (Goldstine's words).

This chapter is a guided tour through 75 years of answers:

- The **floating-point model** and why exact arithmetic is impossibly slow
- **Backward stability**: the right way to measure algorithmic quality
- The **growth factor** ρ: the single number that controls stability
- **Complete vs. partial pivoting**: theory vs. practice
- **Open problems**: what we still don't know after 75 years

Along the way, we'll see Julia benchmarks showing why exact rational arithmetic takes 40 minutes for what floating-point does in 10 milliseconds, and why the growth factor is the "Rosetta Stone" connecting analysis to computation.

1.2 What Is Gaussian Elimination?

Given an invertible matrix $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ and a vector $\boldsymbol{b} \in \mathbb{R}^n$, Gaussian elimination solves $\boldsymbol{Ax} = \boldsymbol{b}$ in two stages:

1. **LU Factorization**: Factor $\boldsymbol{A} = \boldsymbol{LU}$ where $\boldsymbol{L}$ is unit lower-triangular and $\boldsymbol{U}$ is upper-triangular.
2. **Substitution**: Solve $\boldsymbol{Ly} = \boldsymbol{b}$ (forward) then $\boldsymbol{Ux} = \boldsymbol{y}$ (backward).

The second stage is straightforward and numerically stable (conditional on the first). The heart of the algorithm—and the source of all instability—is the LU factorization.

The block-recursive formulation reveals the structure. Partition

$$\boldsymbol{A} = \begin{bmatrix} a_{11} & \boldsymbol{a}_{12}^\top \\ \boldsymbol{a}_{21} & \boldsymbol{A}_{22} \end{bmatrix}, \quad \boldsymbol{L} = \begin{bmatrix} 1 & \boldsymbol{0}^\top \\ \boldsymbol{\ell}_{21} & \boldsymbol{L}_{22} \end{bmatrix}, \quad \boldsymbol{U} = \begin{bmatrix} u_{11} & \boldsymbol{u}_{12}^\top \\ \boldsymbol{0} & \boldsymbol{U}_{22} \end{bmatrix}.$$

If $a_{11} \neq 0$, the first step computes

$$u_{11} = a_{11}, \quad \boldsymbol{u}_{12} = \boldsymbol{a}_{12}, \quad \boldsymbol{\ell}_{21} = \boldsymbol{a}_{21}/a_{11}, \quad \boldsymbol{A}_{22} \leftarrow \boldsymbol{A}_{22} - \boldsymbol{\ell}_{21}\boldsymbol{u}_{12}^\top,$$

where the last line is a **rank-one update** (Schur complement). Recurse on the updated $\boldsymbol{A}_{22}$.

Critical issue: What if $a_{11} = 0$? Or worse, what if a_{11} is *tiny*? Division by a small pivot amplifies rounding errors catastrophically. This is why **pivoting** (row/column permutations) is essential.

1.3 Floating-Point Arithmetic: Why We Can't Use Exact Rational

Example 1.1 (The cost of exactness)**.** *Consider solving a* 1000×1000 *system with random binary entries in Julia:*

Algorithm 1 Gaussian Elimination (no pivoting)

for $k = 1$ to $n - 1$ **do**
  if $A_{kk} = 0$ **then**
    fail
  end if
  for $i = k + 1$ to n **do**
    $L_{ik} \leftarrow A_{ik}/A_{kk}$
  end for
  for $j = k + 1$ to n **do**
    for $i = k + 1$ to n **do**
      $A_{ij} \leftarrow A_{ij} - L_{ik}A_{kj}$
    end for
  end for
end for
$U \leftarrow$ upper triangle of A, $L \leftarrow$ unit lower triangle of A

Listing 1.1: Exact vs. floating-point solve

```
n = 1000
A = rand(Bool, n, n)
b = rand(Bool, n)

# Floating-point (Float64, 53 bits precision)
@time x_fp = Float64.(A) \ Float64.(b)
# 0.01 seconds

# Exact rational (BigInt numerator/denominator)
@time x_exact = Rational{BigInt}.(A) \ Rational{BigInt}.(b)
# 2389.89 seconds (~40 minutes)

# Relative error
norm(x_fp - x_exact) / norm(x_exact)
# 4.34e-13
```

The exact solve takes ***240,000×*** ***longer****. The numerator/denominator of the first entry of* $\boldsymbol{x}_{exact}$ *exceed* ***3200 bits****. For* $n = 2000$*, exact arithmetic would take* weeks*.*

Floating-point is not a bug—it's a necessity. The IEEE 754 model:

$$\mathrm{fl}(x) = x(1 + \delta), \quad |\delta| \leq u,$$

where $u = 2^{-53} \approx 1.1 \times 10^{-16}$ (unit roundoff for double precision). Every operation $\circ \in \{+, -, \times, /\}$ satisfies

$$\mathrm{fl}(a \circ b) = (a \circ b)(1 + \delta), \quad |\delta| \leq u.$$

This **fundamental axiom** is the foundation of all rounding-error analysis.

1.4 Backward Stability: The Right Metric

Suppose an algorithm computes $\hat{\boldsymbol{x}} \approx \boldsymbol{x}$ for $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$. The **forward error** is $\|\hat{\boldsymbol{x}} - \boldsymbol{x}\|$. But if $\boldsymbol{A}$ is ill-conditioned, even the exact solution is sensitive to tiny perturbations in $\boldsymbol{A}$ or $\boldsymbol{b}$—forward error is unfair to the algorithm.

Backward error asks: Does there exist a perturbed problem $(\boldsymbol{A} + \Delta\boldsymbol{A})\hat{\boldsymbol{x}} = \boldsymbol{b} + \Delta\boldsymbol{b}$ with $\|\Delta\boldsymbol{A}\|, \|\Delta\boldsymbol{b}\|$ small, such that $\hat{\boldsymbol{x}}$ is the *exact* solution? If yes, the algorithm is **backward stable**.

For LU factorization, the computed $\hat{\boldsymbol{L}}, \hat{\boldsymbol{U}}$ satisfy

$$\hat{\boldsymbol{L}}\hat{\boldsymbol{U}} = \boldsymbol{A} + \Delta\boldsymbol{A},$$

and the fundamental theorem (Golub & Van Loan, Theorem 3.4.1) gives the bound:

Theorem 1.2 (LU Backward Error)**.** *If Gaussian elimination encounters no zero pivots, the computed factors satisfy*

$$|\Delta\boldsymbol{A}| \leq \gamma_n |\hat{\boldsymbol{L}}||\hat{\boldsymbol{U}}|, \quad \gamma_n = \frac{nu}{1 - nu} \approx nu.$$

Componentwise: the error in each entry is bounded by nu times the corresponding entry of $|\hat{\boldsymbol{L}}||\hat{\boldsymbol{U}}|$.

This transforms a *computational* question into a *mathematical* one: **How large are the entries of $\hat{\boldsymbol{L}}$ and $\hat{\boldsymbol{U}}$ relative to $\boldsymbol{A}$?**

1.5 The Growth Factor: Stability's Rosetta Stone

Definition 1.3 (Growth Factor)**.** *For $\boldsymbol{A} = \boldsymbol{L}\boldsymbol{U}$ (with pivoting $\boldsymbol{P}\boldsymbol{A} = \boldsymbol{L}\boldsymbol{U}$), the **growth factor** is*

$$\rho = \frac{\max_{i,j} |U_{ij}|}{\max_{i,j} |A_{ij}|}.$$

Equivalently, $\rho = \|\boldsymbol{U}\|_{\max} / \|\boldsymbol{A}\|_{\max}$.

The growth factor ρ is the *only* quantity in Theorem 1.2 that depends on the matrix $\boldsymbol{A}$ and the pivoting strategy. Everything else (n, u) is fixed by the problem size and precision.

Corollary 1.4. *Gaussian elimination is backward stable iff ρ is small (say, $\rho = O(n)$ or less).*

Example 1.5 (Worst-case growth without pivoting)**.** *The Wilkinson matrix*

$$\boldsymbol{A} = \begin{bmatrix} 1 & & & & 1 \\ -1 & 1 & & & 1 \\ -1 & -1 & 1 & & 1 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ -1 & -1 & \cdots & -1 & 1 \end{bmatrix}$$

has $\rho = 2^{n-1}$ without pivoting. For $n = 100$, $\rho \approx 10^{30}$—catastrophic.

1.6 Pivoting Strategies

Pivoting permutes rows/columns to control ρ. Three main strategies:

- **Complete pivoting**: At step k, choose $(i^*, j^*) = \text{argmax}_{i,j \geq k} |A_{ij}|$. Swap rows $k \leftrightarrow i^*$ and columns $k \leftrightarrow j^*$. Cost: $O(n^3)$ comparisons (dominates factorization).
- **Partial pivoting**: Choose $i^* = \text{argmax}_{i \geq k} |A_{ik}|$. Swap rows $k \leftrightarrow i^*$. Cost: $O(n^2)$ comparisons (negligible).
- **Rook pivoting**: A compromise—search until a "local maximum" in both row and column.

Theorem 1.6 (Wilkinson 1961: Complete Pivoting Bound)**.** *For complete pivoting,* $\rho \leq n^{1/2}(2 \cdot 3^{1/2} \cdots n^{1/(n-1)})^{1/2} \sim cn^{1/2}(\log n)^{1/2}$.

This proved complete pivoting is stable. But it's *slow*. Partial pivoting is what everyone uses.

Theorem 1.7 (Wilkinson 1961: Partial Pivoting Bound)**.** *For partial pivoting,* $\rho \leq 2^{n-1}$.

This bound is *tight*—the Wilkinson matrix achieves it. **Theoretically, partial pivoting is exponentially unstable.**

Yet in practice, ρ for partial pivoting is almost always tiny (often < 10). This **Wilkinson's paradox** drove decades of research:

- **Higham & Higham (1989)**: Lower bounds on ρ for *any* pivoting strategy.
- **Trefethen & Schreiber (1990)**: Average-case analysis—ρ grows like $n^{2/3}$ for random matrices.
- **Demmel, Grigori, Rusciano (2023)**: Expected $\log \rho$ for random orthogonal transformations.

1.7 Why Partial Pivoting Works in Practice

Example 1.8 (Typical growth factors)**.** *For random matrices (standard normal entries), empirical growth factors with partial pivoting:*

n	*100*	*500*	*1000*	*5000*
ρ *(median)*	*3.2*	*4.1*	*4.8*	*6.2*
ρ *(99th percentile)*	*12*	*28*	*45*	*110*

Nowhere near 2^{n-1}. *The worst-case matrices are* extremely rare *and* highly structured.

Remark 1.9 (What makes a matrix "bad" for partial pivoting?)**.** *Matrices with large growth factors have a special structure: they are nearly* ***upper Hessenberg*** *with a specific pattern of cancellations. Random matrices don't have this structure. In scientific computing, the matrices arising from PDE discretizations, covariance estimation, or structural mechanics are typically well-behaved (often symmetric positive definite, where Cholesky is stable without pivoting).*

1.8 Code: Measuring Growth Factor in Julia

Listing 1.2: Growth factor computation

```
using LinearAlgebra

function growth_factor(A; pivoting=:partial)
    n = size(A, 1)
    A = copy(Float64.(A))
    max_A = maximum(abs, A)

    if pivoting == :none
        # No pivoting LU
        for k = 1:n-1
            for i = k+1:n
                A[i,k] /= A[k,k]
            end
            for j = k+1:n
                for i = k+1:n
                    A[i,j] -= A[i,k] * A[k,j]
                end
            end
        end
    elseif pivoting == :partial
        # Partial pivoting (row swaps)
        for k = 1:n-1
            # Find pivot
            i_max = argmax(abs.(A[k:n, k])) + k - 1
            if i_max != k
                A[k, :], A[i_max, :] = A[i_max, :], A[k, :]
            end
            for i = k+1:n
                A[i,k] /= A[k,k]
            end
            for j = k+1:n
                for i = k+1:n
                    A[i,j] -= A[i,k] * A[k,j]
                end
            end
        end
    end

    U = UpperTriangular(A)
    max_U = maximum(abs, U.data)
    return max_U / max_A
end

# Test
```

```
using Random
Random.seed!(42)
for n in [100, 500, 1000]
    A = randn(n, n)
    rho = growth_factor(A, pivoting=:partial)
    println("n=$n, rho=$rho")
end
```

1.9 Open Problems

Despite 75 years of progress, fundamental questions remain:

1. **Average-case ρ for partial pivoting**: Prove $\mathbb{E}[\rho] \sim n^{2/3}$ (Trefethen-Schreiber conjecture).
2. **Smoothed analysis**: Spielman-Teng style analysis for LU—show that tiny random perturbations make ρ small with high probability.
3. **Communication-avoiding LU**: Stability of CALU and butterfly-based factorizations for modern architectures.
4. **Mixed-precision iterative refinement**: Using FP16/FP32 for factorization, FP64 for refinement (Carson & Higham, 2018).

1.10 Bridge to Chapter 2

Gaussian elimination solves *linear* systems. But many scientific problems—chemical equilibrium, power flow, neural network verification—reduce to *polynomial* systems with *few terms* (fewnomials). Chapter 2 introduces algorithmic fewnomial theory: how the monomial structure determines the number of real solutions and enables efficient solving beyond the reach of generic polynomial methods.

1.11 Further Reading

- Higham, *Accuracy and Stability of Numerical Algorithms* (SIAM, 2002)—the definitive reference.
- Trefethen & Bau, *Numerical Linear Algebra* (SIAM, 1997)—accessible partial pivoting analysis.
- Wilkinson, *The Algebraic Eigenvalue Problem* (Oxford, 1965)—the original breakthrough papers.
- Demmel, Grigori, Rusciano, *Improved bounds for the expected log growth factor* (2023).

1.12 Exercises

Exercise 1.1. Implement complete pivoting in Julia and compare growth factors on the Wilkinson matrix for $n = 20, 50, 100$.

Exercise 1.2. Generate a random 100×100 matrix with condition number 10^{12} (using SVD). Compare forward error of LU with partial pivoting vs. QR factorization.

Exercise 1.3. Prove that for symmetric positive definite $\boldsymbol{A}$, Gaussian elimination without pivoting has $\rho \leq 1$.

Exercise 1.4 (Research)**.** Investigate growth factors for matrices from SuiteSparse (e.g., finite element matrices). Are they ever large?

CHAPTER 2

Algorithmic Fewnomial Theory Over $\mathbb{R}$

J. Maurice Rojas
Communicated by *Notices* Associate Editor Han-Bom Moon

> "There is another family of equations, arguably as simple, that can naturally introduce us to convex geometry and numerical algebraic geometry: those defined by n-variate k-nomials."
>
> — J. Maurice Rojas

Abstract.A **fewnomial** is a polynomial with few monomial terms relative to its degree. While classical algebraic geometry focuses on dense polynomials, fewnomials arise naturally in chemical reaction networks, power systems, neural network verification, and optimization. This chapter develops the theory of fewnomials: root counting via tropical geometry, the Kushnirenko-Bernstein theorem, and algorithmic approaches that exploit sparsity to solve systems far beyond the reach of dense methods.

2.1 Introduction: Why Fewnomials?

In grade school we learn to solve $ax + b = 0$ and $ax^2 + bx + c = 0$. The quadratic formula handles any degree-2 polynomial. But what about

$$x^{100} + 3x^{17} - 5x^3 + 2 = 0?$$

This is a **4-nomial** in 1 variable. It has only 4 terms but degree 100. The fundamental theorem of algebra says 100 complex roots, but how many are *real*? How many are *positive*? Can we find them efficiently?

This is the domain of **fewnomial theory**: polynomials where the number of terms k is much smaller than the degree d. The central questions (S1–S5 from the article):

1. **Existence**: Are there any real roots?
2. **Counting**: How many real roots?

3. **Components**: If infinitely many, how many connected components?
4. **Shape**: Can we classify the topology (homology, isotopy type)?
5. **Approximation**: Can we efficiently produce an ε-net of the zero set?

Why care? Because fewnomials are everywhere:

- **Chemical reaction networks**: Steady states = solutions to sparse polynomial systems (mass-action kinetics). Phosphorylation cascades yield $n \sim 20$, $k \sim 3$ per equation.
- **Power flow equations**: $P_i = \sum_j V_i V_j (G_{ij} \cos\theta_{ij} + B_{ij} \sin\theta_{ij})$ — trigonometric fewnomials.
- **Neural network verification**: ReLU networks are piecewise linear; decision boundaries are fewnomial.
- **Robotics**: Kinematics of linkages $\to$ sparse polynomial systems.

2.2 From Binomials to Fewnomials

Definition 2.1 (k-nomial)**.** *A k-**nomial** in n variables is a polynomial with exactly k nonzero monomial terms:*

$$f(\boldsymbol{x}) = \sum_{j=1}^{k} c_j \boldsymbol{x}^{\boldsymbol{\alpha}_j}, \quad c_j \in \mathbb{R} \setminus \{0\}, \quad \boldsymbol{\alpha}_j \in \mathbb{Z}^n.$$

*The **support** is* $\mathrm{Supp}(f) = \{\boldsymbol{\alpha}_1, \ldots, \boldsymbol{\alpha}_k\} \subset \mathbb{Z}^n$.

Example 2.2 (Binomials are trivial)**.** $f(x) = c_1 x^a + c_2 x^b = 0 \iff x^{a-b} = -c_2/c_1$. *For $x > 0$, this has a solution iff c_1 and c_2 have opposite signs. The positive zero set is either empty or a single point.*

Definition 2.3 (Tropical Variety (Positive))**.** *For $f = \sum_{j=1}^{k} c_j \boldsymbol{x}^{\boldsymbol{\alpha}_j}$ with $c_j \neq 0$, the **positive tropical variety** is*

$$\mathrm{Trop}_+(f) = \{\boldsymbol{y} \in \mathbb{R}^n : \max_j(\langle \boldsymbol{\alpha}_j, \boldsymbol{y} \rangle + \log|c_j|) \text{ is attained at least twice}\}.$$

Theorem 2.4 (Fundamental Approximation Theorem)**.** *Let f be an n-variate k-nomial. Then the zero set $Z_+(f) = \{\boldsymbol{x} \in \mathbb{R}^n_{>0} : f(\boldsymbol{x}) = 0\}$ is either empty or contractible. Moreover, every point of $Z_+(f)$ is within distance $O(1)$ of $\mathrm{Trop}_+(f)$ (after log-change of coordinates).*

This is remarkable: the zero set of a fewnomial is **topologically simple** (contractible components) and **geometrically approximated** by a polyhedral complex (the tropical variety). The tropical variety is a union of at most $\binom{k}{2}$ polyhedral cones—completely independent of the degree!

2.3 The Simplex Case: $k = n + 1$

When $k = n + 1$, the support $\mathcal{A} = \{\boldsymbol{\alpha}_1, \ldots, \boldsymbol{\alpha}_{n+1}\}$ forms a simplex (if affinely independent). This is the simplest nontrivial case.

Theorem 2.5 (Theorem 1.6 in article, specialized)**.** *For an n-variate $(n+1)$-nomial f with affinely independent support:*

1. *$Z_+(f)$ is either empty or a single contractible component.*
2. *$Z_+(f) \neq \emptyset \iff$ the coefficients c_j do* not *all have the same sign.*
3. *$\mathrm{Trop}_+(f)$ is a single affine hyperplane (when nonempty).*

Example 2.6 (Approximating radicals)**.** *Consider $x^d - a = 0$ for $a > 0$. The positive root is $x = a^{1/d}$. The binomial $x^d - a$ has trop variety at $\log x = \frac{1}{d} \log a$, exactly the root. For a 3-nomial $x^d + c_1 x^a - c_2 = 0$, the tropical variety gives a piecewise-linear approximation to the root that is metrically close.*

2.4 Systems of Fewnomials: Kushnirenko-Bernstein

For a system $f_1 = \cdots = f_n = 0$ in n variables, let $\mathcal{A}_i = \mathrm{Supp}(f_i)$ and $Q_i = \mathrm{conv}(\mathcal{A}_i) \subset \mathbb{R}^n$ be the Newton polytopes.

Theorem 2.7 (Kushnirenko-Bernstein)**.** *The number of isolated solutions in $(\mathbb{C}^*)^n$ (complex torus) is exactly the* ***mixed volume*** $\mathrm{MV}(Q_1, \ldots, Q_n)$*, which equals*

$$n! \cdot \mathrm{Vol}_n(Q_1 + \cdots + Q_n) - \sum_i n! \cdot \mathrm{Vol}_n(Q_1 + \cdots + \widehat{Q_i} + \cdots + Q_n) + \cdots$$

For fewnomial systems, the Newton polytopes have few vertices, making mixed volume computation tractable.

Example 2.8 (Chemical reaction network)**.** *A phosphorylation cycle with 3 species yields a 3-variate system where each equation is a 4-nomial. The Newton polytopes are tetrahedra. Mixed volume = 3 (vs. Bézout bound $4^3 = 64$). The sparse structure reduces the root count by 20x.*

2.5 Arithmetic and Bit Complexity

Solving fewnomial systems requires controlling the **bit size** of solutions.

Problem 2.9 (Approximating radicals)**.** *Given $a, b \in \mathbb{N}$, find $x \in \mathbb{Q}$ with $|x - a^{1/b}| < \varepsilon$.*

Lemma 2.10 (Lemma 1.11 in article)**.** *There is an algorithm using $O(\log b)$ field operations and inequality decisions.*

Theorem 2.11 (Theorem 1.12, BMST)**.** *Computing $a^{1/b}$ to accuracy ε requires $\Omega(\log b + \log(1/\varepsilon))$ arithmetic operations. This is* asymptotically optimal*.*

The key insight: **Newton's method on binomials** achieves the lower bound. For fewnomials, the tropical approximation provides a **near-optimal initial guess**, after which local refinement (Newton/homotopy) converges rapidly.

2.6 Code: Tropical Approximation for Fewnomials

Listing 2.1: Tropical variety of a fewnomial

```
import numpy as np
from itertools import combinations

def tropical_variety_positive(alpha, c):
    """
    alpha: k x n array of exponent vectors
    c: k array of coefficients
    Returns polyhedral cones (as halfspace intersections) for Trop_+(f)
    """
    k, n = alpha.shape
    logc = np.log(np.abs(c))
    signs = np.sign(c)

    cones = []
    for (i, j) in combinations(range(k), 2):
        # Equality of two terms: alpha_i^T y + log|c_i| = alpha_j^T y + log|c_j|
        # => (alpha_i - alpha_j)^T y = log|c_j| - log|c_i|
        A_eq = alpha[i] - alpha[j]
        b_eq = logc[j] - logc[i]

        # Dominance over all other terms
        A_ub = []
        b_ub = []
        for l in range(k):
            if l == i or l == j: continue
            A_ub.append(alpha[l] - alpha[i])
            b_ub.append(logc[i] - logc[l])
             logc[l])
        cones.append((A_eq, b_eq, np.array(A_ub), np.array(b_ub)))
    return cones

def solve_tropical(alpha, c):
    """Find a point in Trop_+(f) via linear programming"""
    from scipy.optimize import linprog
    cones = tropical_variety_positive(alpha, c)
    for A_eq, b_eq, A_ub, b_ub in cones:
        # Minimize 0 subject to A_eq @ y = b_eq, A_ub @ y <= b_ub
        res = linprog(np.zeros(A_eq.shape[0]), A_ub=A_ub, b_ub=b_ub,
                      A_eq=A_eq.reshape(1,-1), b_eq=[b_eq], method='highs')
        if res.success:
            return res.x
    return None

# Example: f(x,y) = x^3 + 2*x*y - 5*y^2
alpha = np.array([[3,0], [1,1], [0,2]])
c = np.array([1, 2, -5])
```

```
y = solve_tropical(alpha, c)
print("Tropical point (log coords):", y)
print("Approx root (exp):", np.exp(y))
```

2.7 Applications

- **Steady states of chemical reaction networks**: Fewnomial systems from mass-action kinetics. The **deficiency zero theorem** (Feinberg) guarantees uniqueness of positive steady states when the network structure is a directed forest.
- **Power flow**: The AC power flow equations are trigonometric fewnomials. Converting via $z = e^{i\theta}$ gives polynomial fewnomials. Homotopy methods with tropical start systems solve large grids.
- **Neural network verification**: The decision boundary of a ReLU network is a piecewise linear function. The number of linear regions is a fewnomial count (related to hyperplane arrangements).
- **Polynomial optimization**: Minimizing a fewnomial over a polytope reduces to checking critical points (fewnomial system) and boundary.

2.8 Bridge to Chapter 3

Fewnomial theory uses **Newton polytopes** and **mixed volumes** to count roots. Chapter 3 uses **spectra of matrices** (eigenvalues) to understand networks. Both are "spectral" in a broad sense: fewnomials use the *combinatorial spectrum* (exponent vectors), spectral methods use the *algebraic spectrum* (eigenvalues). Both exploit sparsity/structure to bypass dense complexity.

2.9 Further Reading

- Rojas, *Algorithmic Fewnomial Theory* (this article, Notices 2025).
- Khovanskii, *Fewnomials* (AMS Translations, 1991)—the foundational monograph.
- Sturmfels, *Solving Systems of Polynomial Equations* (CBMS, 2002)—tropical geometry.
- Bihan, Sottile, *New bounds for fewnomials* (2007).
- Feinberg, *Foundations of Chemical Reaction Network Theory* (Springer, 2019).

2.10 Exercises

Exercise 2.1. Compute the tropical variety of $f(x, y, z) = x^2 + 2y^3 - 3z + 4$. How many polyhedral cones?

Exercise 2.2. For the phosphorylation cycle (3 species, each equation a 4-nomial), write the system and compute its mixed volume using `PHCpack` or `HomotopyContinuation.jl`.

Exercise 2.3. Show that a univariate k-nomial has at most $k-1$ positive real roots (Descartes' rule of signs). What about negative roots?

Exercise 2.4 (Research)**.** Implement a homotopy continuation solver that uses tropical varieties as start systems for 3-variate 4-nomial systems. Compare to random start systems.

CHAPTER 3

Spectral Methods in Microeconomics

Benjamin Golub
Communicated by *Notices* Associate Editor Daniela De Silva

> "Matrices often appear in formal models of social and economic behavior... Spectral theory offers powerful tools for understanding matrices, and economic modelers have leveraged these tools to gain considerable insight."
>
> — Benjamin Golub

Abstract. Eigenvalues and eigenvectors are the hidden skeleton of networked systems. This chapter shows how Perron-Frobenius theory, Katz-Bonacich centrality, and spectral decomposition unify opinion dynamics, network games, and market equilibrium. The same mathematics describes how beliefs spread, how firms compete, and how central banks should act.

3.1 Introduction: Why Spectral Methods?

In microeconomics, agents interact: consumers learn from neighbors, firms compete on networks, banks lend to each other. These interactions are encoded in **matrices**. The spectral properties of these matrices—especially the dominant eigenvalue and eigenvector—determine the system's long-run behavior.

Consider a row-stochastic matrix $\boldsymbol{W}$ (nonnegative, rows sum to 1) representing social influence. The DeGroot model of opinion dynamics:

$$\boldsymbol{x}(t+1) = \boldsymbol{W}\boldsymbol{x}(t).$$

If $\boldsymbol{W}$ is irreducible and aperiodic, $\boldsymbol{x}(t) \to \boldsymbol{\pi}^\top \boldsymbol{x}(0)\mathbf{1}$ where $\boldsymbol{\pi}$ is the **left Perron vector** (stationary distribution). Convergence rate is governed by $|\lambda_2|$, the second-largest eigenvalue modulus.

This single fact unifies:

- **Opinion dynamics**: How fast do beliefs converge?
- **Centrality**: Who is influential? (Katz-Bonacich = Perron vector)

- **Network games**: Equilibrium actions = $(\boldsymbol{I} - \beta \boldsymbol{G})^{-1}\boldsymbol{a}$; spectral radius of $\beta \boldsymbol{G}$ determines uniqueness.
- **Information aggregation**: "Wisdom of crowds" iff $\max_i \pi_i \to 0$ as $n \to \infty$.

3.2 Perron-Frobenius Theory for Economists

Theorem 3.1 (Perron-Frobenius (Irreducible Nonnegative)). *Let $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, $a_{ij} \geq 0$, irreducible. Then:*

1. *$\rho(\boldsymbol{A}) > 0$ is a simple eigenvalue (the **Perron root**).*
2. *There exist $\boldsymbol{u} > 0$, $\boldsymbol{v} > 0$ (right/left Perron vectors) with $\boldsymbol{A}\boldsymbol{u} = \rho \boldsymbol{u}$, $\boldsymbol{v}^\top \boldsymbol{A} = \rho \boldsymbol{v}^\top$.*
3. *All other eigenvalues satisfy $|\lambda| < \rho$.*
4. *For any $\boldsymbol{x} > 0$, $\boldsymbol{A}^t \boldsymbol{x}/\rho^t \to \boldsymbol{u}(\boldsymbol{v}^\top \boldsymbol{x}/\boldsymbol{v}^\top \boldsymbol{u})$.*

Definition 3.2 (Irreducibility). *$\boldsymbol{A}$ is **irreducible** iff its directed graph is strongly connected: for any i, j, there is a path $i \to j$. Equivalently, no permutation makes $\boldsymbol{A}$ block upper-triangular.*

In economics, irreducibility = "the network is connected." If the economy has isolated sectors, spectral theory applies block by block.

3.3 Application 1: DeGroot Learning and Social Influence

Agents update beliefs by averaging neighbors:

$$x_i(t+1) = \sum_j w_{ij} x_j(t), \quad \boldsymbol{W} \text{ row-stochastic.}$$

Proposition 3.3. *If $\boldsymbol{W}$ is irreducible and aperiodic, $\boldsymbol{x}(t) \to \boldsymbol{\pi}^\top \boldsymbol{x}(0)\mathbf{1}$ where $\boldsymbol{\pi}$ is the unique stationary distribution ($\boldsymbol{\pi}^\top \boldsymbol{W} = \boldsymbol{\pi}^\top$, $\boldsymbol{\pi} > 0$, $\sum \pi_i = 1$).*

The **social influence** of agent i is π_i. This is exactly **eigenvector centrality** (left Perron vector of $\boldsymbol{W}$).

Example 3.4 (Star network). *Agent 1 at center, all others peripheral. $\boldsymbol{W}_{1j} = 1/(n-1)$, $\boldsymbol{W}_{i1} = 1$ for $i > 1$. Then $\pi_1 = 1/2$, $\pi_i = 1/(2n-2)$ for $i > 1$. The center has influence $\sim 1/2$ regardless of n!*

Definition 3.5 (Wisdom of Crowds). *A sequence of networks is **wise** if $\max_i \pi_i^{(n)} \to 0$. Golub-Jackson (2010): wisdom holds iff no agent has "disproportionate influence."*

3.4 Application 2: Network Games and Bonacich Centrality

Agents choose actions $a_i \in \mathbb{R}$. Payoff:

$$u_i(a_i, \boldsymbol{a}_{-i}) = a_i \left(\alpha_i - \frac{1}{2} a_i + \beta \sum_j g_{ij} a_j \right).$$

Best response: $a_i = \alpha_i + \beta \sum_j g_{ij} a_j$. In vector form:

$$\boldsymbol{a} = \boldsymbol{\alpha} + \beta \boldsymbol{G}\boldsymbol{a} \quad \Rightarrow \quad \boldsymbol{a}^* = (\boldsymbol{I} - \beta \boldsymbol{G})^{-1} \boldsymbol{\alpha},$$

provided $\rho(\beta \boldsymbol{G}) < 1$.

Definition 3.6 (Katz-Bonacich Centrality)**.** *For $\boldsymbol{G} \geq 0$ and $\beta < 1/\rho(\boldsymbol{G})$,*

$$\boldsymbol{b}(\beta, \boldsymbol{G}) = (\boldsymbol{I} - \beta \boldsymbol{G})^{-1}\mathbf{1} = \sum_{k=0}^{\infty} \beta^k \boldsymbol{G}^k \mathbf{1}.$$

Entry b_i counts weighted walks starting at i; longer walks discounted by β^k.

The equilibrium action is $\boldsymbol{a}^* = \boldsymbol{b}(\beta, \boldsymbol{G}) \odot \boldsymbol{\alpha}$ (if $\boldsymbol{\alpha}$ is scalar). **Equilibrium actions are proportional to centrality.**

3.5 Application 3: Wisdom, Centralization, and Policy

The Golub-Jackson **prominent sequence** concept: a family of networks $(\boldsymbol{W}^{(n)})$ with n agents. They are wise iff the most influential agent's weight $\to 0$.

Theorem 3.7 (Wisdom Criterion)**.** $(\boldsymbol{W}^{(n)})$ *is wise iff* $\max_i \pi_i^{(n)} \to 0$*. For symmetric $\boldsymbol{W}$, this is equivalent to $\lambda_2(\boldsymbol{W}) \to 1$ (the spectral gap closes).*

Policy insight: if a regulator wants to improve information aggregation, they should **reduce the spectral gap**—i.e., make the network less centralized. This contradicts the intuition that "more connectivity is always better." A star network is highly connected but unwise; a line network is poorly connected but wise.

3.6 Code: Spectral Analysis of Networks

Listing 3.1: Network spectral analysis

```julia
using LinearAlgebra, SparseArrays, LightGraphs
using Statistics

function analyze_network(W; tol=1e-12)
    n = size(W, 1)

    # Check irreducibility
    G = DiGraph(W .> tol)
    is_irred = is_strongly_connected(G)

    # Perron-Frobenius (left eigenvector for row-stochastic)
    eig = eigen(W')
    idx = argmax(abs.(eig.values))
    lambda1 = real(eig.values[idx])
    pi = real(eig.vectors[:, idx])
    pi = pi / sum(pi)  # normalize

    # Second eigenvalue modulus
    lambda2 = maximum(abs.(eig.values[setdiff(1:end, idx)]))
    spectral_gap = lambda1 - lambda2

    # Katz-Bonacich centrality for beta < 1/rho(G)
    # Assuming G is the adjacency (not stochastic)
    function katz_centrality(G, beta)
```

```
        rho = maximum(abs.(eigvals(G)))
        if beta >= 1/rho
            error("beta too large: beta*rho = $(beta*rho)")
        end
        (I - beta*G) \ ones(n)
    end

    # Wisdom metrics
    max_influence = maximum(pi)
    herfindahl = sum(pi.^2)  # concentration index

    return (
        lambda1 = lambda1,
        lambda2 = lambda2,
        spectral_gap = spectral_gap,
        pi = pi,
        max_influence = max_influence,
        herfindahl = herfindahl,
        is_irred = is_irred
    )
end

# Example: Star network
n = 10
W = zeros(n,n)
W[1, 2:end] .= 1/(n-1)
for i = 2:n
    W[i, 1] = 1.0
end

res = analyze_network(W)
println("Star network: max_influence = ", res.max_influence)
println("Spectral gap = ", res.spectral_gap)

# Example: Line network
W2 = zeros(n,n)
for i = 1:n
    neighbors = []
    i > 1 && push!(neighbors, i-1)
    i < n && push!(neighbors, i+1)
    W2[i, neighbors] .= 1/length(neighbors)
end
res2 = analyze_network(W2)
println("Line network: max_influence = ", res2.max_influence)
println("Spectral gap = ", res2.spectral_gap)
```

3.7 Bridge to Chapter 4

Spectral methods analyze **linear** network dynamics: $\boldsymbol{x}(t+1) = \boldsymbol{W}\boldsymbol{x}(t)$. Chapter 4 introduces **nonlinear** network dynamics: deep neural networks. Remarkably, ReLU networks are *piecewise linear*—on each linear region, they are exactly a matrix multiplication. The geometry of deep learning is the geometry of how these linear regions tessellate the input space.

3.8 Further Reading

- Golub, B. (2025). *Spectral Methods in Microeconomics*. Notices of the AMS, 72(6).
- Golub, B., Jackson, M.O. (2010). *Naive Learning in Social Networks and the Wisdom of Crowds*. American Economic Journal: Microeconomics.
- Ballester, C., Calvó-Armengol, A., Zenou, Y. (2006). *Who's Who in Networks. Wanted: The Key Player*. Econometrica.
- Jackson, M.O. (2010). *Social and Economic Networks*. Princeton University Press.
- Bonacich, P. (1987). *Power and Centrality: A Family of Measures*. American Journal of Sociology.

3.9 Exercises

Exercise 3.1. For the star network with n agents, compute the exact stationary distribution π and verify $\pi_1 = 1/2$ for all $n \geq 3$.

Exercise 3.2. Show that for a row-stochastic matrix $\boldsymbol{W}$, the Katz-Bonacich centrality with $\boldsymbol{G} = \boldsymbol{W}$ and $\beta < 1$ is $\boldsymbol{b} = (\boldsymbol{I} - \beta \boldsymbol{W})^{-1}\mathbf{1}$. What does this represent in the DeGroot model?

Exercise 3.3. Implement the Golub-Jackson wisdom test: generate a sequence of Erdős-Rényi graphs $G(n, p_n)$ with $p_n = \log n / n$ and $p_n = 1/\sqrt{n}$. Which sequence is wise?

Exercise 3.4 (Research)**.** Extend the network game to heterogeneous β_i (different interaction strengths). When does a unique equilibrium exist?

Part II

Geometric Machine Learning

CHAPTER 4

On the Geometry of Deep Learning

Randall Balestriero, Ahmed Imtiaz Humayun, Richard G. Baraniuk
Communicated by *Notices* Associate Editor Reza Malek-Madani

> "Deep networks are affine spline operators. They tessellate input space into convex polytopes, and on each tile they act as an affine transformation."
>
> — Balestriero, Humayun, Baraniuk

Abstract. This chapter develops a geometric theory of deep ReLU networks: they are **affine spline mappings** that partition input space into convex polytopes (tiles) and apply a distinct affine transformation on each. This perspective explains generalization, batch normalization, grokking, and generative model bias—and yields practical tools like SplineCam for visualization and MaGNET for debiasing.

4.1 Introduction: The Black Box Problem

Deep networks work astonishingly well but remain poorly understood. The standard view: black boxes optimized by SGD. This chapter proposes a **white-box** view:

A deep ReLU network = an affine spline operator.

- Input space $\mathbb{R}^d$ is tiled into convex polytopes $\Omega = \{\Omega_j\}$.
- On each tile Ω_j, the network computes $\boldsymbol{x} \mapsto \boldsymbol{A}_j\boldsymbol{x} + \boldsymbol{b}_j$.
- The tiles and transforms depend on weights/biases and change during training.

This is not an approximation—it is an *exact* mathematical equivalence for any finite-width ReLU network.

4.2 Affine Splines in 1D

A 1D continuous piecewise linear function (affine spline) is defined by:

- Knots: $t_1 < t_2 < \cdots < t_K$
- Slopes: $m_0, m_1, \ldots, m_K$
- Intercepts: $b_0, b_1, \ldots, b_K$

with continuity constraints $m_i t_{i+1} + b_i = m_{i+1} t_{i+1} + b_{i+1}$.

A ReLU layer with n neurons computes

$$\boldsymbol{y} = \mathrm{ReLU}(\boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}), \quad \boldsymbol{W} \in \mathbb{R}^{n \times d}.$$

Each neuron i creates a hyperplane $\{\boldsymbol{x} : \boldsymbol{w}_i^\top \boldsymbol{x} + b_i = 0\}$ dividing space into two halfspaces. The arrangement of n hyperplanes creates a tessellation into convex polytopes.

4.3 Deep Network Tessellation

Theorem 4.1 (Deep Network = Affine Spline)**.** *For a depth-L ReLU network $f : \mathbb{R}^d \to \mathbb{R}^m$, there exists a finite tessellation $\Omega = \{\Omega_j\}_{j=1}^N$ of $\mathbb{R}^d$ into convex polytopes and affine maps $\boldsymbol{A}_j \in \mathbb{R}^{m \times d}, \boldsymbol{b}_j \in \mathbb{R}^m$ such that*

$$f(\boldsymbol{x}) = \boldsymbol{A}_j \boldsymbol{x} + \boldsymbol{b}_j \quad \textit{for all } \boldsymbol{x} \in \Omega_j.$$

The polytopes are the linear regions *of the network.*

Sketch. Induction on layers. Base: one ReLU layer creates a hyperplane arrangement (convex polytopes). Inductive step: each subsequent layer pulls back its hyperplanes through the previous layer's affine maps, which *folds* the existing tiles, creating a finer tessellation. The affine map on each tile composes with the new layer's affine map. □

Example 4.2 (Number of tiles)**.** *For a 2D input, 2-hidden-layer network with widths 20, the tessellation can have $\sim 10^4$ tiles. For deep networks, the number grows exponentially with depth (Montúfar et al., 2014).*

4.4 Geometry of the Loss Landscape

For squared error $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2}\|f_{\boldsymbol{\theta}}(\boldsymbol{X}) - \boldsymbol{Y}\|_F^2$, the loss landscape as a function of parameters $\boldsymbol{\theta}$ is a **piecewise quadratic function**. On each region where the input-space tessellation is constant, $\mathcal{L}$ is quadratic in $\boldsymbol{\theta}$.

Theorem 4.3 (ResNet vs. ConvNet conditioning)**.** *The condition number of the local Hessian for a ResNet (with skip connections) is* bounded *independent of depth, while for a plain ConvNet it* grows exponentially *with depth.*

This explains why ResNets train faster and more reliably: their loss landscape is better conditioned.

4.5 Batch Normalization as Data-Adaptive Tessellation

BatchNorm replaces $\boldsymbol{x}^{(\ell)} \mapsto \boldsymbol{W}^{(\ell)}\boldsymbol{x}^{(\ell)} + \boldsymbol{b}^{(\ell)}$ with

$$\boldsymbol{x}^{(\ell)} \mapsto \boldsymbol{\gamma} \odot \frac{\boldsymbol{W}^{(\ell)}\boldsymbol{x}^{(\ell)} - \boldsymbol{\mu}}{\boldsymbol{\sigma}} + \boldsymbol{\beta},$$

where $\boldsymbol{\mu}, \boldsymbol{\sigma}$ are batch statistics.

Theorem 4.4 (BatchNorm Geometry)**.** *At initialization, BatchNorm* adapts the hyperplane arrangement *to concentrate tile boundaries near the training data. Without BatchNorm, tiles are randomly oriented; with BatchNorm, hyperplanes align to data density.*

This is why BatchNorm accelerates training: it gives the network a *data-aware initialization* of its tessellation.

4.6 The Dynamics of Learning: Grokking

Grokking (Power et al., 2022): A network trained well past zero training error suddenly improves test accuracy—the tessellation *migrates* from overfitting (many small tiles around training points) to generalizing (fewer, larger tiles focused on decision boundaries).

Definition 4.5 (Local Complexity)**.** *For a point $\boldsymbol{x}$ in input space, the **local complexity** is the number of tiles intersecting a neighborhood $\mathcal{N}_\epsilon(\boldsymbol{x})$.*

During grokking, local complexity *decreases* around training data and *increases* at decision boundaries.

Listing 4.1: SplineCam: Visualizing network tessellation

```python
import torch
import numpy as np
from splinecam import SplineCam

# Load trained model
model = torch.load('mnist_mlp.pt')

# Compute tessellation on a 2D slice defined by 3 training points
sc = SplineCam(model)
slice_2d = sc.compute_slice(points=[img1, img2, img3], grid_size=200)

# Visualize
sc.plot_tessellation(slice_2d, color_by='tile_id')
sc.plot_affine_maps(slice_2d)
sc.plot_decision_boundary(slice_2d)
```

4.7 Generative Models: MaGNET Debiasing

A ReLU generative model $G : \mathbb{R}^k \to \mathbb{R}^d$ maps a low-dimensional latent space to a *piecewise affine manifold* in data space. Each latent tile Ω_j maps affinely to a manifold tile $G(\Omega_j)$.

Theorem 4.6 (Volume distortion)**.** *The volume scaling factor from latent tile Ω_j to data tile $G(\Omega_j)$ is $|\det \boldsymbol{A}_j|$ where $\boldsymbol{A}_j$ is the affine map's linear part.*

MaGNET (Balestriero et al., 2022): To sample uniformly from the data manifold, sample latent points with probability $\propto 1/|\det \boldsymbol{A}_j|$. This corrects for the *mode collapse* and *bias* of standard uniform latent sampling.

Listing 4.2: MaGNET debiasing for StyleGAN

```python
import torch
from magnett import magnet_sampler

# Load pretrained StyleGAN2
G = load_stylegan2('ffhq.pkl')

# Compute Jacobian determinants on a grid
latent_grid = torch.randn(10000, 512)
with torch.no_grad():
    jac_dets = batch_jacobian_det(G, latent_grid)

# MaGNET importance weights
weights = 1.0 / (jac_dets + 1e-8)
weights = weights / weights.sum()

# Sample debiased latents
debiased_latents = latent_grid[torch.multinomial(weights, 1000)]

# Generate fair samples
fair_images = G(debiased_latents)
```

4.8 Code: Affine Spline Layer

Listing 4.3: Exact affine spline representation of a ReLU layer

```julia
using LinearAlgebra, SparseArrays

struct AffineSplineLayer
    tiles::Vector{HPolyhedron}  # Convex polytopes (H-representation)
    A::Vector{Matrix{Float64}}  # Linear maps per tile
    b::Vector{Vector{Float64}}  # Biases per tile
end

function relu_layer_to_spline(W, b)
    n, d = size(W)
    tiles = HPolyhedron[]
    A_list = Matrix{Float64}[]
    b_list = Vector{Float64}[]

    # Enumerate all 2^n sign patterns
    for signs in Iterators.product(fill([-1, 1], n)...)
        # sign_i = 1 means w_i^T x + b_i >= 0 (active)
        # sign_i = -1 means w_i^T x + b_i <= 0 (inactive)
        H = []
```

```
        h = []
        for i in 1:n
            if signs[i] == 1
                push!(H, W[i, :])
                push!(h, b[i])
            else
                push!(H, -W[i, :])
                push!(h, -b[i])
            end
        end
        # Check feasibility (non-empty polytope)
        if is_feasible(H, h)
            poly = HPolyhedron(hcat(H...), h)
            push!(tiles, poly)

            # Affine map on this tile: y = D(Wx + b) where D = diag(max(
                sign, 0))
            D = Diagonal([s == 1 ? 1.0 : 0.0 for s in signs])
            push!(A_list, D * W)
            push!(b_list, D * b)
        end
    end
    return AffineSplineLayer(tiles, A_list, b_list)
end

function evaluate_spline(spline::AffineSplineLayer, x)
    for (i, tile) in enumerate(spline.tiles)
        if x in tile
            return spline.A[i] * x + spline.b[i]
        end
    end
    error("Point not in any tile")
end
```

4.9 Bridge to Chapter 5

Deep networks are affine splines. Equivariant networks (Chapter 5) constrain these splines to respect symmetries: the tessellation and affine maps must commute with group actions. Topological deep learning (Chapter 6) uses invariants like the Euler characteristic to classify the shapes these splines produce.

4.10 Further Reading

- Balestriero, Humayun, Baraniuk (2025). *On the Geometry of Deep Learning*. Notices, 72(4).
- Montúfar et al. (2014). *On the Number of Linear Regions of Deep Neural Networks*. NIPS.
- Balestriero et al. (2022). *MaGNET: Maximum Entropy Generative Network*. ICML.
- SplineCam: https://github.com/randall-b/splinecam
- MaGNET: https://github.com/randall-b/magnet

4.11 Exercises

Exercise 4.1. For a 1-hidden-layer ReLU network with 3 neurons in 2D, draw the hyperplane arrangement and label the affine map on each tile.

Exercise 4.2. Implement the exact affine spline representation of a 2-layer ReLU network. Verify that evaluating the spline matches the network output for random inputs.

Exercise 4.3. Use SplineCam to visualize the tessellation of a trained MNIST classifier. How does the tile structure differ between correctly and incorrectly classified examples?

Exercise 4.4 (Research)**.** Prove that the number of linear regions of a deep ReLU network is upper-bounded by $\prod_{\ell=1}^{L} \sum_{j=0}^{d} \binom{n_\ell}{j}$ where n_ℓ is the width of layer ℓ.

CHAPTER 5

What Is... an Equivariant Neural Network?

Lek-Heng Lim, Bradley J. Nelson
Communicated by *Notices* Associate Editor Emilie Purvine

> "An equivariant neural network is an equivariant map constructed from alternately composing equivariant linear maps with nonlinear ones."
>
> — Lim & Nelson

Abstract. Symmetry is everywhere in science: molecular structures, physical laws, images. Standard neural networks ignore symmetry, wasting capacity learning what physics already dictates. Equivariant neural networks bake symmetry into the architecture: if the input transforms, the output transforms predictably. This chapter gives the mathematical foundation: group representations, intertwining operators, and the universal approximation theorem for equivariant networks.

5.1 Introduction: Why Symmetry Matters

A molecule rotated in space is the *same* molecule. A cat translated in an image is the same cat. The laws of physics are translation/rotation invariant. Yet a standard CNN must *learn* translation invariance from data—a waste of parameters and samples.

An **equivariant neural network** guarantees:

$$f(g \cdot x) = g \cdot f(x) \quad \forall g \in G,$$

where G is a symmetry group acting on input space $\mathcal{X}$ and output space $\mathcal{Y}$.

- **Invariance**: $f(g \cdot x) = f(x)$ (e.g., classification: rotated cat $\to$ "cat")
- **Equivariance**: $f(g \cdot x) = g \cdot f(x)$ (e.g., segmentation: rotated input $\to$ rotated mask)
- **Covariance**: Output transforms by a *different* representation of G.

5.2 Group Representations

Definition 5.1 (Group Representation)**.** *A **representation** of a group G on a vector space V is a homomorphism $\rho : G \to \mathrm{GL}(V)$. For $g \in G$, $\rho(g)$ is an invertible linear map on V.*

Key examples for deep learning:

- **Translation** ($G = \mathbb{Z}^2$ or $\mathbb{R}^2$): $\rho(g)_{ij} = x_{i-g}$ (shift)
- **Rotation** ($G = \mathrm{SO}(2)$ or $\mathrm{SO}(3)$): $\rho(g) = R_g$ (rotation matrix)
- **Permutation** ($G = S_n$): $\rho(\sigma)_{ij} = \delta_{i,\sigma(j)}$
- **Permutation + Sign** ($G = S_n \times \{\pm 1\}^n$): for fermionic wavefunctions

Theorem 5.2 (Schur's Lemma)**.** *Let $\rho_1 : G \to \mathrm{GL}(V_1)$, $\rho_2 : G \to \mathrm{GL}(V_2)$ be irreducible representations. Any linear map $T : V_1 \to V_2$ satisfying $T\rho_1(g) = \rho_2(g)T$ for all g is either zero or an isomorphism. If $V_1 = V_2$ and $\rho_1 = \rho_2$ irreducible, then $T = \lambda I$.*

Consequence: Equivariant linear layers are highly constrained! For irreps, they are just scalar multiples of the identity.

5.3 Equivariant Linear Layers

We want linear $\boldsymbol{W} : V_1 \to V_2$ such that $\boldsymbol{W}\rho_1(g) = \rho_2(g)\boldsymbol{W}$.

Example 5.3 (Convolution = Translation Equivariance)**.** *Let $G = \mathbb{Z}^2$ act on images by translation. An equivariant linear map is exactly a **convolution**:*

$$(\boldsymbol{W} * x)[i] = \sum_j w[i-j]x[j].$$

Proof: $(T_g x)[i] = x[i-g]$. *Then* $(T_g \boldsymbol{W} * x)[i] = \sum_j w[i-j]x[j-g] = (\boldsymbol{W} * T_g x)[i]$.

Theorem 5.4 (General Solution for Linear Equivariant Maps)**.** *For finite G, the space of equivariant linear maps between representations ρ_1, ρ_2 is*

$$\mathrm{Hom}_G(\rho_1, \rho_2) = \left\{ \frac{1}{|G|} \sum_{g \in G} \rho_2(g) M \rho_1(g^{-1}) : M \in \mathbb{R}^{d_2 \times d_1} \right\}.$$

This is the projection *onto the equivariant subspace (Reynolds operator).*

5.4 Equivariant Nonlinearities

Linear equivariance is well understood. Nonlinearities break equivariance unless carefully chosen.

Proposition 5.5. *A pointwise nonlinearity $\sigma : \mathbb{R} \to \mathbb{R}$ is G-equivariant iff it commutes with the action of G on $\mathbb{R}$.*

For permutation groups S_n acting by permuting coordinates, **any** pointwise σ is equivariant (since $\sigma(x_{\pi(i)}) = \sigma(x)_{\pi(i)}$). This is why ReLU works in standard MLPs and CNNs!

For rotation groups $\mathrm{SO}(3)$, pointwise σ on vector components is *not* equivariant (rotation mixes components). Solutions:

1. **Scalarization**: Apply σ to *invariants* (norms, dot products)
2. **Gated nonlinearities**: $\sigma(\boldsymbol{x}) = \sigma_{\text{scalar}}(\|\boldsymbol{x}\|) \cdot \boldsymbol{x}/\|\boldsymbol{x}\|$
3. **Tensor product**: Work in irreps; Clebsch-Gordan coefficients give equivariant polynomial maps

5.5 Architecture: Equivariant Neural Networks

Definition 5.6 (Equivariant Neural Network)**.** *A feedforward network where:*

1. *Each layer* $\boldsymbol{x}^{(\ell+1)} = \sigma(\boldsymbol{W}^{(\ell)}\boldsymbol{x}^{(\ell)} + \boldsymbol{b}^{(\ell)})$
2. $\boldsymbol{W}^{(\ell)}$ *is G-equivariant (intertwiner)*
3. σ *is G-equivariant (pointwise on scalars, gated on vectors)*
4. $\boldsymbol{b}^{(\ell)}$ *is G-invariant (typically zero or learned invariant features)*

The composition of equivariant maps is equivariant, so the whole network is equivariant.

Example 5.7 (SO(3)-Equivariant Network for Molecules)**.**

1. *Input: atomic positions* $\boldsymbol{r}_i \in \mathbb{R}^3$*, types* z_i
2. *Embed types as scalars (invariant features)*
3. *Spherical harmonics* $Y_l^m(\boldsymbol{r}_i - \boldsymbol{r}_j)$ *as equivariant features (transform as* $D^l(R)$*)*
4. *Tensor product layers using Clebsch-Gordan coefficients*
5. *Gated nonlinearities preserve equivariance*
6. *Output: scalar (energy), vector (forces), tensor (stress)*

This is the architecture behind ***NequiP***, ***MACE***, ***SE(3)-Transformer***.

5.6 Code: Implementing an Equivariant Layer

Listing 5.1: SO(2)-equivariant convolution using steerable filters

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from math import factorial

class SO2Conv(nn.Module):
    """
    Steerable CNN for SO(2) equivariance.
    Features transform as Fourier modes: f_l(r, theta) = sum_m f_{l,m}(r) e^{im theta}
    """
    def __init__(self, in_channels, out_channels, max_freq, kernel_size=3):
        super().__init__()
```

```
        self.max_freq = max_freq
        self.in_channels = in_channels
        self.out_channels = out_channels

        # Learnable radial profiles for each frequency
        # For each input freq l_in and output freq l_out, we need a radial kernel
        # l_out = l_in + m where m is the filter's frequency
        self.radial_mlps = nn.ModuleDict()

        for l_in in range(max_freq + 1):
            for l_out in range(max_freq + 1):
                m = l_out - l_in  # filter frequency
                if abs(m) <= max_freq:
                    key = f"{l_in}_{l_out}"
                    # Radial profile: small MLP taking radius as input
                    self.radial_mlps[key] = nn.Sequential(
                        nn.Linear(1, 16), nn.ReLU(),
                        nn.Linear(16, in_channels * out_channels)
                    )

    def forward(self, x):
        # x: [B, C_in, L_max+1, 2, H, W]
        #   complex features: real/imag for each frequency
        B, C_in, L, _, H, W = x.shape

        # Convert to polar coordinates for convolution
        # This is a simplified version; real implementation uses FFT
        out = torch.zeros(B, self.out_channels, self.max_freq+1, 2, H, W, device=x.device)

        for l_in in range(self.max_freq + 1):
            for l_out in range(self.max_freq + 1):
                key = f"{l_in}_{l_out}"
                if key not in self.radial_mlps:
                    continue

                # Get radial weights
                # In practice, we evaluate radial profile on grid and do convolution via FFT
                pass  # Full implementation requires careful FFT handling

        return out

# Simpler: Scalar + Vector fields for SO(3)
class SO3Conv(nn.Module):
    """Tensor field network style: scalars (l=0) and vectors (l=1)"""
    def __init__(self, in_scalar, in_vector, out_scalar, out_vector):
        super().__init__()
        # Scalar -> Scalar: standard conv
        self.ss = nn.Conv2d(in_scalar, out_scalar, 3, padding=1)
        # Scalar -> Vector: multiply by learned vector field
        self.sv = nn.Conv2d(in_scalar, out_vector * 3, 3, padding=1)
        # Vector -> Scalar: dot product with learned vector
```

```
        self.vs = nn.Conv2d(in_vector * 3, out_scalar, 3, padding=1)
        # Vector -> Vector: linear combination + cross product
        self.vv = nn.Conv2d(in_vector * 3, out_vector * 3, 3, padding=1)

    def forward(self, scalars, vectors):
        # scalars: [B, C_s, H, W]
        # vectors: [B, C_v, 3, H, W]  (3 components)
        B, C_v, _, H, W = vectors.shape
        v_flat = vectors.view(B, C_v * 3, H, W)

        out_s = self.ss(scalars) + self.vs(v_flat)
        out_v_flat = self.sv(scalars) + self.vv(v_flat)
        out_v = out_v_flat.view(B, -1, 3, H, W)

        return out_s, out_v
```

5.7 Universal Approximation

Theorem 5.8 (Equivariant Universal Approximation)**.** *Let G be a compact group. An equivariant neural network with:*

- *Equivariant linear layers spanning* $\mathrm{Hom}_G(\rho_1, \rho_2)$
- *Equivariant nonlinearities separating orbits*

can approximate any continuous G-equivariant function $f : V_1 \to V_2$ uniformly on compact sets.

Key insight: The nonlinearity must *not* be G-invariant (e.g., ReLU on scalars works for S_n but not SO(3)). Gated nonlinearities provide the needed separation.

5.8 Bridge to Chapter 6

Equivariant networks constrain the **symmetry** of the affine spline tessellation (Chapter 4). Topological deep learning (Chapter 6) studies the **shape** of the decision boundaries and feature spaces using algebraic topology invariants. Both are structural priors: symmetry reduces parameters; topology measures complexity.

5.9 Further Reading

- Lim, Nelson (2023). *What Is... an Equivariant Neural Network?* Notices, 70(4).
- Cohen, Welling (2016). *Group Equivariant CNNs.* ICML.
- Thomas et al. (2018). *Tensor Field Networks.* NeurIPS.
- Fuchs et al. (2020). *SE(3)-Transformers.* ICML.
- Geiger, Smidt (2022). *e3nn: Euclidean Neural Networks.* https://github.com/e3nn/e3nn

5.10 Exercises

Exercise 5.1. Show that a standard CNN with ReLU is translation-equivariant but not rotation-equivariant.

Exercise 5.2. Implement the Reynolds operator for $G = D_4$ (dihedral group of square) acting on 2×2 images. Verify it projects any matrix onto the D_4-equivariant subspace.

Exercise 5.3. For $\mathrm{SO}(2)$ acting on $\mathbb{R}^2$ by rotation, find all equivariant linear maps $\mathbb{R}^2 \to \mathbb{R}^2$. What about $\mathbb{R}^2 \to \mathbb{R}$?

Exercise 5.4 (Research)**.** Design an equivariant architecture for the group $G = \mathrm{SE}(3)$ (rigid motions) acting on point clouds. Compare to PointNet++ on ModelNet40.

CHAPTER 6

Topology Meets Machine Learning: The Euler Characteristic Transform

Bastian Rieck
Communicated by *Notices* Associate Editor Emilie Purvine

> "The Euler Characteristic Transform serves as a sufficient shape statistic, yielding an injective mapping for finite geometric simplicial complexes."
>
> — Bastian Rieck

Abstract. The Euler Characteristic Transform (ECT) converts a shape (simplicial complex) into a function on the sphere: for each direction, filter the shape by height and compute the Euler characteristic. The ECT is invertible, differentiable (via sigmoid approximation), and provides a topological inductive bias for deep learning. This chapter introduces the ECT, its mathematical foundations, and applications to shape classification and point cloud reconstruction.

6.1 Introduction: Topology as Inductive Bias

Deep learning excels at learning features from data. But what if we already know something about the *structure* of the data? For shapes (3D meshes, point clouds, graphs), topology provides invariants that are robust to deformation, noise, and discretization.

The **Euler Characteristic Transform (ECT)** bridges topology and machine learning:

- Input: a geometric simplicial complex $K \subset \mathbb{R}^d$
- **Topological**: Based on Euler characteristic $\chi = \beta_0 - \beta_1 + \beta_2 - \cdots$
- **Multiscale**: Filtration by direction gives a curve, not just a number
- **Complete**: The ECT is injective (Turner et al., 2014; Ghrist et al., 2018)
- **Differentiable**: Sigmoid approximation enables gradient-based learning
- **Efficient**: $O(\text{faces} \times \text{directions})$, trivial to parallelize

6.2 The Euler Characteristic

Definition 6.1 (Euler Characteristic)**.** *For a finite simplicial complex K with f_k simplices of dimension k:*

$$\chi(K) = \sum_{k=0}^{d} (-1)^k f_k = \sum_{k=0}^{d} (-1)^k \beta_k.$$

β_k are Betti numbers (ranks of homology groups). χ is a homotopy invariant.

Example 6.2 (Platonic Solids)**.** *All five Platonic solids are homeomorphic to S^2, so $\chi = 2$.*

	Vertices	*Edges*	*Faces*
Tetrahedron	*4*	*6*	*4*
Cube	*8*	*12*	*6*
Octahedron	*6*	*12*	*8*
Dodecahedron	*20*	*30*	*12*
Icosahedron	*12*	*30*	*20*

$V - E + F = 2$ for all.

6.3 The Euler Characteristic Transform

Let $K \subset \mathbb{R}^d$ be a geometric simplicial complex. For a direction $\boldsymbol{v} \in S^{d-1}$ and threshold $t \in \mathbb{R}$, define the **sublevel set**

$$K_{\boldsymbol{v},t} = \{\sigma \in K : \max_{\boldsymbol{x} \in \sigma} \langle \boldsymbol{v}, \boldsymbol{x} \rangle \leq t\}.$$

Definition 6.3 (Euler Characteristic Curve (ECC))**.** *The ECC in direction $\boldsymbol{v}$ is the function*

$$\mathrm{ECC}_K(\boldsymbol{v}, t) = \chi(K_{\boldsymbol{v},t}).$$

Definition 6.4 (Euler Characteristic Transform (ECT))**.** *The ECT is the map*

$$\mathrm{ECT}_K : S^{d-1} \to \{S^{d-1} \to \mathbb{Z}\}, \quad \boldsymbol{v} \mapsto \mathrm{ECC}_K(\boldsymbol{v}, \cdot).$$

It assigns to each direction its Euler Characteristic Curve.

Theorem 6.5 (Invertibility of ECT)**.** *For finite geometric simplicial complexes in $\mathbb{R}^d$, the ECT is injective:* $\mathrm{ECT}_K = \mathrm{ECT}_L \implies K = L$. *Moreover, a* finite *number of directions suffices (Curry et al., 2022).*

The proof uses Euler calculus: the ECT is a **Radon transform** of the indicator function, invertible by the Euler integral.

6.4 Discretization and Machine Learning

For computation, we discretize:

1. **Directions**: Sample D directions $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_D \in S^{d-1}$.
2. **Thresholds**: For each direction, sample T thresholds $t_1 < \cdots < t_T$.

The ECT becomes a $D \times T$ matrix (image). Each column is an ECC.

Listing 6.1: Discretized ECT computation

```python
import numpy as np
from scipy.spatial import Delaunay

def compute_ect(vertices, faces, directions, thresholds):
    """
    vertices: N x 3 array
    faces: M x 3 array of vertex indices (triangles)
    directions: D x 3 array (unit vectors)
    thresholds: T array of heights
    Returns: D x T matrix of Euler characteristics
    """
    D, T = len(directions), len(thresholds)
    ect = np.zeros((D, T), dtype=int)

    for d, v in enumerate(directions):
        # Project vertices onto direction
        heights = vertices @ v  # N array

        for t_idx, t in enumerate(thresholds):
            # Vertices below threshold
            v_below = heights <= t

            # Edges below threshold
            edges = set()
            for face in faces:
                for i, j in [(0,1), (1,2), (2,0)]:
                    edges.add(tuple(sorted((face[i], face[j]))))
            edges = np.array(list(edges))
            e_below = np.all(v_below[edges], axis=1)
            f1 = np.sum(e_below)

            # Faces below threshold
            f_below = np.all(v_below[faces], axis=1)
            f2 = np.sum(f_below)

            f0 = np.sum(v_below)
            ect[d, t_idx] = f0 - f1 + f2

    return ect

# Example: Tetrahedron
verts = np.array([[1,1,1], [-1,-1,1], [-1,1,-1], [1,-1,-1]]) / np.sqrt(3)
faces = np.array([[0,1,2], [0,1,3], [0,2,3], [1,2,3]])
directions = np.random.randn(100, 3)
directions = directions / np.linalg.norm(directions, axis=1, keepdims=True
    )
thresholds = np.linspace(-1, 1, 50)

ect_matrix = compute_ect(verts, faces, directions, thresholds)
print("ECT shape:", ect_matrix.shape)  # (100, 50)
```

6.5 Differentiable ECT for Deep Learning

The ECC is a step function (changes only when threshold crosses a vertex height). To use in deep learning, we need a **smooth approximation**.

Proposition 6.6 (Sigmoid Approximation)**.** *Replace the indicator* $\mathbb{1}_{h \le t}$ *with* $\sigma_\beta(t-h) = (1 + e^{-\beta(t-h)})^{-1}$*. Then*

$$\widetilde{\mathrm{ECC}}_K(\boldsymbol{v}, t) = \sum_{\sigma \in K} (-1)^{\dim \sigma} \prod_{\boldsymbol{x} \in \sigma} \sigma_\beta(t - \langle \boldsymbol{v}, \boldsymbol{x} \rangle).$$

As $\beta \to \infty$, $\widetilde{\mathrm{ECC}} \to \mathrm{ECC}$ *pointwise.*

This makes the ECT a **differentiable layer** in a neural network. The input is a point cloud/mesh; the output is an ECT matrix; gradients flow back to vertex positions.

6.6 Applications

1. **Shape Classification**: ECT matrix as feature vector → SVM or CNN. Competitive with graph neural networks on geometric graphs (Rieck et al., 2024).
2. **Point Cloud Reconstruction**: Learn vertex positions by minimizing ECT distance to target (Rieck, 2025).
3. **Topological Regularization**: Add ECT loss to enforce shape priors in generative models.
4. **Protein Shape Analysis**: ECT of molecular surfaces for binding site prediction.

Listing 6.2: ECT-based shape classification

```python
import torch
import torch.nn as nn

class ECTClassifier(nn.Module):
    def __init__(self, n_directions=100, n_thresholds=50, n_classes=10):
        super().__init__()
        self.ect_layer = DifferentiableECT(n_directions, n_thresholds)
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(n_directions * n_thresholds, 256),
            nn.ReLU(),
            nn.Linear(256, n_classes)
        )

    def forward(self, vertices, faces):
        # vertices: [B, N, 3], faces: [B, M, 3]
        ect = self.ect_layer(vertices, faces)  # [B, D, T]
        return self.classifier(ect)

# Differentiable ECT layer (using sigmoid approximation)
class DifferentiableECT(nn.Module):
    def __init__(self, n_directions, n_thresholds, beta=10.0):
        super().__init__()
        # Random directions on sphere
```

```
        dirs = torch.randn(n_directions, 3)
        self.register_buffer('directions', dirs / dirs.norm(dim=1, keepdim=True))
        self.register_buffer('thresholds', torch.linspace(-1, 1, n_thresholds))
        self.beta = beta

    def forward(self, vertices, faces):
        B, N, _ = vertices.shape
        D, T = len(self.directions), len(self.thresholds)

        ect = torch.zeros(B, D, T, device=vertices.device)

        for d, v in enumerate(self.directions):
            heights = vertices @ v  # [B, N]

            for t_idx, t in enumerate(self.thresholds):
                # Sigmoid approximation for vertices
                v_below = torch.sigmoid(self.beta * (t - heights))  # [B, N]
                f0 = v_below.sum(dim=1)  # [B]

                # For edges/faces, need to multiply sigmoids
                # Simplified: this is a sketch; real implementation uses batched ops
                # ... (full implementation computes edges/faces similarly)

                ect[:, d, t_idx] = f0  # + edge/face terms

        return ect
```

6.7 Code Repository

- Python package: `pip install ect` (https://github.com/bastianrieck/ect)
- Differentiable ECT in PyTorch: https://github.com/bastianrieck/diffect
- Tutorial notebooks: https://topology.rocks/ect

6.8 Bridge to Chapter 7

The ECT provides a **topological summary** of shape. Chapter 7 addresses a different kind of summary: **uncertainty quantification**. Instead of describing a shape, UQ describes what we *don't know* about a model's predictions. Both are "compression" operations: ECT compresses geometry into curves; UQ compresses probability distributions into entropy/KL divergence.

6.9 Further Reading

- Rieck (2025). *Topology Meets Machine Learning: An Introduction Using the Euler Characteristic Transform.* Notices, 72(7).

- Turner et al. (2014). *Euler Integral Transforms.* J. Appl. Comput. Topol.
- Ghrist et al. (2018). *Persistent Homology and Euler Integral Transforms.* J. Appl. Comput. Topol.
- Curry et al. (2022). *How Many Directions Determine a Shape?* Trans. AMS.
- Munch (2025). *The Euler Characteristic Transform.* Notices (overview).

6.10 Exercises

Exercise 6.1. Compute the ECT of a cube with vertices at $(\pm 1, \pm 1, \pm 1)$ for directions $(\pm 1, 0, 0)$, $(0, \pm 1, 0)$, $(0, 0, \pm 1)$. Verify $\chi = 2$ for all thresholds beyond the cube extent.

Exercise 6.2. Implement the exact (non-differentiable) ECT for a triangle mesh. Compare the ECT matrices of a sphere and a torus. How many directions are needed to distinguish them?

Exercise 6.3. Use the differentiable ECT to reconstruct a point cloud of a chair from its ECT matrix. Initialize with random points and optimize via gradient descent.

Exercise 6.4 (Research)**.** Prove that the ECT of a simplicial complex determines its persistent homology (or vice versa).

Part III

Uncertainty, Dynamics, and Inverse Problems

CHAPTER 7

Taming Uncertainty: The Rise of Uncertainty Quantification

Nan Chen, Stephen Wiggins, Marios Andreou
Communicated by *Notices* Associate Editor Reza Malek-Madani

> "All models are wrong, but some are useful."
>
> — George Box

Abstract. Uncertainty Quantification (UQ) provides a systematic framework for characterizing how unknowns propagate through mathematical models. This chapter is a self-contained tutorial: from Shannon entropy and KL divergence through data assimilation (Kalman/Ensemble filters) to stochastic surrogate modeling for turbulent PDEs. Complete with MATLAB/Python code for every algorithm.

7.1 Introduction: Why Quantify Uncertainty?

Mathematical models have imperfections:

- **Parametric uncertainty**: Unknown coefficients (e.g., viscosity, reaction rates)
- **Structural uncertainty**: Missing physics, simplified equations
- **Initial/boundary condition uncertainty**: Noisy or sparse observations
- **Numerical uncertainty**: Discretization, rounding errors

UQ asks: *How do these uncertainties affect the model output?* The answer is a probability distribution (or at least its moments) on the output space.

Two fundamental information measures:

Shannon Entropy $H(p) = -\int p(x) \log p(x) dx$ — spread of a single distribution

KL Divergence $D_{\mathrm{KL}}(p\|q) = \int p(x) \log \frac{p(x)}{q(x)} dx$ — difference between two distributions

7.2 Entropy and Relative Entropy

For a Gaussian $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ in d dimensions:

$$H = \frac{1}{2}\log\det(2\pi e\boldsymbol{\Sigma}) = \frac{d}{2}(1+\log 2\pi) + \frac{1}{2}\log\det\boldsymbol{\Sigma}.$$

For non-Gaussian distributions, entropy captures features variance cannot:

- **Skewness**: Asymmetric tails → higher entropy
- **Multimodality**: Multiple peaks → higher entropy
- **Heavy tails**: Extreme events → higher entropy

Example 7.1 (Gamma vs. Gaussian with same variance)**.** Gamma$(2,1)$ *has variance 2, entropy* ≈ 1.68. $\mathcal{N}(0,2)$ *has entropy* ≈ 1.42. *Same variance, different uncertainty!*

KL divergence measures the cost of using approximate model q when truth is p:

$$D_{\mathrm{KL}}(p\|q) = \underbrace{\frac{1}{2}\mathrm{tr}(\boldsymbol{\Sigma}_q^{-1}\boldsymbol{\Sigma}_p) - \frac{d}{2} + \frac{1}{2}\log\frac{\det\boldsymbol{\Sigma}_q}{\det\boldsymbol{\Sigma}_p}}_{\text{Dispersion}} + \underbrace{\frac{1}{2}(\boldsymbol{\mu}_p-\boldsymbol{\mu}_q)^\top\boldsymbol{\Sigma}_q^{-1}(\boldsymbol{\mu}_p-\boldsymbol{\mu}_q)}_{\text{Signal}}.$$

7.3 Uncertainty in Dynamical Systems

Consider $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x})$ with uncertain initial condition $\boldsymbol{x}(0) \sim p_0$.

Linear systems: $\dot{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{x}$. If $\boldsymbol{A}$ is stable, uncertainty *decays* and mean dynamics separate:

$$\frac{d}{dt}\mathbb{E}[\boldsymbol{x}] = \boldsymbol{A}\mathbb{E}[\boldsymbol{x}], \quad \frac{d}{dt}\mathrm{Cov}[\boldsymbol{x}] = \boldsymbol{A}\mathrm{Cov}[\boldsymbol{x}] + \mathrm{Cov}[\boldsymbol{x}]\boldsymbol{A}^\top.$$

Nonlinear systems: Reynolds decomposition $\boldsymbol{x} = \bar{\boldsymbol{x}} + \boldsymbol{x}'$:

$$\frac{d}{dt}\bar{\boldsymbol{x}} = \mathbb{E}[\boldsymbol{f}(\bar{\boldsymbol{x}}+\boldsymbol{x}')] \neq \boldsymbol{f}(\bar{\boldsymbol{x}}).$$

The *mean dynamics couple to higher moments.* For quadratic $\boldsymbol{f}$:

$$\mathbb{E}[\boldsymbol{x}\boldsymbol{x}^\top] = \bar{\boldsymbol{x}}\bar{\boldsymbol{x}}^\top + \mathrm{Cov}[\boldsymbol{x}].$$

The covariance feeds back into the mean! This is the **closure problem**.

Chaotic systems (Lorenz-63): Small initial uncertainty → exponential divergence (positive Lyapunov exponents). Ensemble mean quickly becomes meaningless.

7.4 Data Assimilation: Kalman and Ensemble Filters

Combine model forecast $\boldsymbol{x}^f \sim \mathcal{N}(\boldsymbol{\mu}^f, \boldsymbol{P}^f)$ with observation $\boldsymbol{y} = \boldsymbol{H}\boldsymbol{x} + \boldsymbol{\eta}$, $\boldsymbol{\eta} \sim \mathcal{N}(0, \boldsymbol{R})$.

Theorem 7.2 (Kalman Filter Update)**.** *Posterior (analysis) distribution:*

$$\boldsymbol{\mu}^a = \boldsymbol{\mu}^f + \boldsymbol{K}(\boldsymbol{y} - \boldsymbol{H}\boldsymbol{\mu}^f), \quad \boldsymbol{P}^a = (\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\boldsymbol{P}^f,$$

where $\boldsymbol{K} = \boldsymbol{P}^f\boldsymbol{H}^\top(\boldsymbol{H}\boldsymbol{P}^f\boldsymbol{H}^\top + \boldsymbol{R})^{-1}$ *is the* ***Kalman gain****.*

For nonlinear/high-dimensional systems, use **Ensemble Kalman Filter (EnKF)**:

1. Maintain ensemble $\{\boldsymbol{x}_i^f\}_{i=1}^N$ representing forecast distribution
2. Sample observations $\boldsymbol{y}_i = \boldsymbol{y} + \boldsymbol{\eta}_i$, $\boldsymbol{\eta}_i \sim \mathcal{N}(0, \boldsymbol{R})$
3. Update each member: $\boldsymbol{x}_i^a = \boldsymbol{x}_i^f + \boldsymbol{K}(\boldsymbol{y}_i - \boldsymbol{H}\boldsymbol{x}_i^f)$
4. $\boldsymbol{K}$ estimated from ensemble covariance

Listing 7.1: Ensemble Kalman Filter

```julia
using LinearAlgebra, Statistics, Random

function enkf_update(ensemble::Matrix, H, y, R)
    # ensemble: n_state x n_ensemble
    # H: n_obs x n_state
    # y: n_obs observation
    # R: n_obs x n_obs observation noise covariance
    n_state, N = size(ensemble)

    # Ensemble mean and perturbation matrix
    mu_f = mean(ensemble, dims=2)
    X_f = ensemble .- mu_f  # n_state x N

    # Forecast ensemble in observation space
    Y_f = H * ensemble  # n_obs x N
    y_mean = mean(Y_f, dims=2)
    Y_pert = Y_f .- y_mean

    # Observation perturbations
    y_obs = y .+ sqrt(R) * randn(n_obs, N)

    # Ensemble covariances
    Pf_xy = (X_f * Y_pert') / (N - 1)  # n_state x n_obs
    Pf_yy = (Y_pert * Y_pert') / (N - 1)  # n_obs x n_obs

    # Kalman gain
    K = Pf_xy * inv(Pf_yy + R)

    # Analysis update
    ensemble_a = ensemble + K * (y_obs - Y_f)
    return ensemble_a
end

# Lorenz-63 twin experiment
function lorenz63(u, p, t)
    sigma, rho, beta = 10.0, 28.0, 8/3
    x, y, z = u
    [sigma*(y-x), x*(rho-z)-y, x*y - beta*z]
end

# Run EnKF on Lorenz-63
using DifferentialEquations
prob = ODEProblem(lorenz63, [1.0, 1.0, 1.0], (0.0, 10.0))
```

```julia
true_sol = solve(prob, Tsit5(), saveat=0.01)

# Generate synthetic observations (every 0.1 time units, observe x only)
obs_times = 0.0:0.1:10.0
obs = true_sol(obs_times)[1, :] + 0.5 * randn(length(obs_times))

# EnKF assimilation
ensemble = [1.0, 1.0, 1.0] .+ 0.5 * randn(3, 50)  # 50 members
H = [1.0 0.0 0.0]
R = [0.25]

for (i, t) in enumerate(obs_times)
    # Forecast step
    prob = ODEProblem(lorenz63, ensemble, (t, t+0.1))
    ensemble = hcat([solve(prob, Tsit5(), saveat=0.1).u[end] for _ in
       1:50]...)

    # Analysis step
    ensemble = enkf_update(ensemble, H, obs[i], R)
end
```

7.5 Stochastic Surrogate Modeling

High-dimensional PDEs (climate, turbulence) are too expensive for ensemble forecasting. Replace with a stochastic model that matches statistics.

Consider a Fourier mode $\hat{u}_k(t)$ from a turbulent PDE:

$$\frac{d\hat{u}_k}{dt} = \lambda_k \hat{u}_k + \sum_{p+q=k} N_{k,p,q} \hat{u}_p \hat{u}_q + F_k.$$

Stochastic surrogate: Replace nonlinear sum with noise calibrated to match statistics:

$$\frac{d\hat{u}_k}{dt} = \lambda_k \hat{u}_k + \sigma_k \dot{W}_k(t),$$

where σ_k chosen so equilibrium PDF and decorrelation time match the true mode.

Theorem 7.3 (Calibration)**.** *For Ornstein-Uhlenbeck $dx = -\theta x dt + \sigma dW$:*

- *Equilibrium variance:* $\sigma^2/(2\theta) = \mathrm{Var}_{true}$
- *Decorrelation time:* $1/\theta = \tau_{true}$

Solve for θ, σ.

Example 7.4 (2-Layer Quasi-Geostrophic Model)**.** *Full model:* 128^2 *modes per layer. Stochastic surrogate: only 5 leading modes, calibrated to their statistics. Qualitatively similar spatial fields, matching temporal correlations and PDFs.*

7.6 Bridge to Chapter 8

UQ quantifies uncertainty in *dynamical systems*. Chapter 8 applies algebraic geometry to a *static* but nonlinear problem: the global positioning equations. Both use the same insight: *reformulate the problem to reveal its mathematical structure* (sphere intersection for GPS, entropy/KL for UQ).

7.7 Further Reading

- Chen, Wiggins, Andreou (2025). *Taming Uncertainty....* Notices, 72(3). Code: https://github.com/marandmath/UQ_tutorial_code
- Chen (2023). *Stochastic Methods for Modeling and Predicting Complex Dynamical Systems.* Springer.
- Law, Stuart, Zygalakis (2015). *Data Assimilation.* Springer.
- Reich, Cotter (2015). *Probabilistic Forecasting and Bayesian Data Assimilation.* Cambridge.
- Majda, Harlim (2012). *Filtering Complex Turbulent Systems.* Science.

7.8 Exercises

Exercise 7.1. Compute the KL divergence between $\mathcal{N}(0,1)$ and $\mathcal{N}(0.5,1.5)$. Interpret the signal and dispersion terms.

Exercise 7.2. Implement the EnKF for the Lorenz-96 system (40 variables, observe every 4th). Compare RMSE vs. ensemble size $N = 10, 20, 50, 100$.

Exercise 7.3. For a scalar OU process with true variance 2.0 and decorrelation time 0.5, compute the surrogate parameters θ, σ. Simulate and verify equilibrium statistics.

Exercise 7.4 (Research)**.** Apply the stochastic surrogate approach to the Kuramoto-Sivashinsky equation. Can you reduce the number of modes by 90

CHAPTER 8

New Developments in Global Positioning

Mireille Boutin, Gregor Kemper
Communicated by *Notices* Associate Editor

> "This is a story about how mathematics can make a difference in a real-world problem... The fundamental question of when the problem has a unique solution and when it does not was still unanswered."
>
> — Boutin & Kemper

Abstract. The Global Positioning System (GPS) solves a system of quadratic equations every time your phone finds its location. This chapter shows how algebraic geometry and Minkowski geometry reveal the exact conditions for unique solvability: the satellite positions must not lie on a quadric with the user position as focus. The solution is a linear algebra problem in disguise.

8.1 Introduction: The Problem

Your phone receives signals from GPS satellites. Each satellite i broadcasts:

- Its position $\boldsymbol{s}_i \in \mathbb{R}^3$ (known precisely)
- Transmission time t_i^{tx} (from atomic clock)

Your phone receives at time t_i^{rx} (from its cheap quartz clock). The *apparent flight time* is $\tau_i = t_i^{\text{rx}} - t_i^{\text{tx}}$.

The **global positioning equations**:

$$\|\boldsymbol{x} - \boldsymbol{s}_i\| = c\tau_i - b, \quad i = 1, \ldots, m$$

where:

- $\boldsymbol{x} \in \mathbb{R}^3$ = your position (unknown)

- b = your clock bias (unknown)
- c = speed of light (set $c = 1$ by time units)
- m = number of satellites in view (≥ 4)

This is a system of m quadratic equations in 4 unknowns.

8.2 Sphere Intersection in Minkowski Space

Square both sides:

$$\|\boldsymbol{x} - \boldsymbol{s}_i\|^2 = (\tau_i - b)^2.$$

Expand:

$$\|\boldsymbol{x}\|^2 - 2\boldsymbol{s}_i^\top \boldsymbol{x} + \|\boldsymbol{s}_i\|^2 = \tau_i^2 - 2\tau_i b + b^2.$$

Introduce the Minkowski inner product on $\mathbb{R}^4$:

$$\langle (\boldsymbol{x}, b), (\boldsymbol{y}, c) \rangle_M = \boldsymbol{x}^\top \boldsymbol{y} - bc.$$

Define $\boldsymbol{X} = (\boldsymbol{x}, b)$, $\boldsymbol{S}_i = (\boldsymbol{s}_i, \tau_i)$, $\sigma_i = \frac{1}{2}(\|\boldsymbol{s}_i\|^2 - \tau_i^2)$. Then:

$$\|\boldsymbol{X}\|_M^2 - 2\langle \boldsymbol{X}, \boldsymbol{S}_i \rangle_M = -2\sigma_i,$$

where $\|\cdot\|_M^2 = \langle \cdot, \cdot \rangle_M$.

This says: $\boldsymbol{X}$ lies on a **Minkowski sphere** (quadric) centered at $\boldsymbol{S}_i$ with "radius" determined by σ_i.

Theorem 8.1 (Sphere Intersection as Linear Algebra)**.** *The system is equivalent to:*

$$\boldsymbol{A}\boldsymbol{X} = \boldsymbol{\sigma},$$

where $\boldsymbol{A}$ has rows $(2\boldsymbol{s}_i^\top, -2\tau_i, -1)$ and $\boldsymbol{\sigma}_i = \|\boldsymbol{s}_i\|^2 - \tau_i^2$.

If $\boldsymbol{A}$ has full column rank (4), then $\boldsymbol{X}$ is uniquely determined by the linear system. Then check $\|\boldsymbol{X}\|_M^2 = \boldsymbol{X}^\top \boldsymbol{\eta} \boldsymbol{X}$ consistency.

8.3 When Is the Solution Unique?

Theorem 8.2 (Boutin-Kemper 2024)**.** *Assume $m \geq 4$ satellites with non-coplanar positions. The GPS problem has:*

- *A **unique** solution iff the satellites do* not *lie on a quadric surface with focus at the user position.*
- ***Two** solutions iff they do lie on such a quadric.*

The quadric is a sphere, ellipsoid, paraboloid, or hyperboloid (depending on eccentricity).

Sketch. If $\boldsymbol{A}$ has rank 3, the linear system gives a line of solutions. Substituting into the quadratic constraint $\|\boldsymbol{X}\|_M^2 = \cdots$ gives a quadratic equation: 0, 1, or 2 solutions. Rank deficiency of $\boldsymbol{A}$ occurs exactly when satellites lie on a quadric with focus $\boldsymbol{X}$. □

Example 8.3 (Practical geometry)**.**

- ***Good**: Satellites well-spread across sky (rank 4)*
- ***Bad**: All satellites in a plane (e.g., urban canyon) → rank 3 → ambiguity*
- ***Bad**: Satellites on a sphere centered at you → quadric with focus at you → two solutions*

8.4 Algorithm: Exact Solution

Algorithm 2 GPS Solution (Exact Data)

procedure GPS-Solve($\boldsymbol{s}_i, \tau_i$ for $i = 1..m$)
  Form $\boldsymbol{A} \in \mathbb{R}^{m\times 4}$ with rows $(2\boldsymbol{s}_i^\top, -2\tau_i, -1)$
  Form $\boldsymbol{\sigma} \in \mathbb{R}^m$ with $\sigma_i = \|\boldsymbol{s}_i\|^2 - \tau_i^2$
  if rank$(\boldsymbol{A}) = 4$ **then**
    $\boldsymbol{X} \leftarrow (\boldsymbol{A}^\top\boldsymbol{A})^{-1}\boldsymbol{A}^\top\boldsymbol{\sigma}$ ▷ Least squares if $m > 4$
    return $\boldsymbol{x} = \boldsymbol{X}_{1:3}, b = \boldsymbol{X}_4$
  else ▷ rank 3
    Find line $\boldsymbol{X} = \boldsymbol{X}_0 + t\boldsymbol{v}$ solving $\boldsymbol{AX} = \boldsymbol{\sigma}$
    Substitute into $\|\boldsymbol{X}\|_M^2 - 2\langle \boldsymbol{X}, \boldsymbol{S}_i\rangle_M + 2\sigma_i = 0$
    Solve quadratic for t (0, 1, or 2 solutions)
    return all valid solutions
  end if
end procedure

8.5 Noisy Data: Weighted Least Squares

Real measurements have noise. The linearized system $\boldsymbol{AX} = \boldsymbol{\sigma}$ has errors from:

- Satellite position errors (small, ∼cm)
- Clock/timing errors (larger, ∼meters)
- Atmospheric delays (ionosphere/troposphere)

Satellite positions are known much more accurately than apparent flight times. So invert the submatrix $\boldsymbol{A}_1$ obtained by deleting the *first* column (the $\boldsymbol{s}_i$ terms), rather than the last column (Bancroft's method).

Listing 8.1: GPS solver with noise

```
using LinearAlgebra, Statistics

function solve_gps(sat_positions::Matrix, tau::Vector; weights=nothing)
    # sat_positions: 3 x m matrix of satellite positions
    # tau: m vector of apparent flight times
    m = length(tau)

    # Build A matrix: rows are [2*s_i', -2*tau_i, -1]
    A = [2*sat_positions'  -2*tau  -ones(m)]

    # sigma = ||s_i||^2 - tau_i^2
    sigma = sum(sat_positions.^2, dims=1)' - tau.^2

    if rank(A) == 4
        if weights === nothing
            X = (A' * A) \ (A' * sigma)
        else
```

```
            W = Diagonal(weights)
            X = (A' * W * A) \ (A' * W * sigma)
        end
        return X[1:3], X[4]
    else
        # Rank 3: find line of solutions
        # A has nullspace of dimension 1
        F = svd(A)
        v_null = F.V[:, end]

        # Particular solution
        X0 = A \ sigma

        # Quadratic constraint: ||X||_M^2 - 2*<X, S_i>_M + 2*sigma_i = 0
        # This gives a quadratic in t: X = X0 + t * v_null
        # ... (details omitted for brevity)
        # Return both solutions if they exist
        error("Rank-deficient case: implement quadratic solve")
    end
end

# Test with simulated data
Random.seed!(42)
true_x = [1000.0, 2000.0, 500.0]  # user position
true_b = 50.0  # clock bias

# 6 satellites
sat = randn(3, 6) * 20000 .+ [0, 0, 20000]  # roughly in orbit
tau = [norm(sat[:,i] - true_x) + true_b for i in 1:6]

# Add noise
tau_noisy = tau + 0.01 * randn(6)

x_est, b_est = solve_gps(sat, tau_noisy)
println("True: ", true_x, " ", true_b)
println("Est:  ", x_est, " ", b_est)
println("Error: ", norm(x_est - true_x))
```

8.6 Conditioning and Geometry

The condition number of the linear system $\boldsymbol{AX} = \boldsymbol{\sigma}$ blows up as satellites approach a quadric with focus at the user. This is *not* just a numerical issue—it reflects a fundamental geometric ambiguity.

Example 8.4 (Urban canyon)**.** *Satellites visible only along a street: positions nearly coplanar. The matrix $\boldsymbol{A}$ has rank 3. Two possible positions symmetric across the satellite plane. The linear system is ill-conditioned; small timing errors → large position errors.*

8.7 Bridge to Chapter 9

GPS solves a *static* nonlinear system by algebraic geometry. Chapter 9 addresses a *dynamic* inverse problem: DNA storage systems where the channel introduces insertions, deletions, and

substitutions—requiring coding theory for edit-distance channels.

8.8 Further Reading

- Boutin, Kemper (2025). *New Developments in Global Positioning*. Notices, 72(8).
- Bancroft (1985). *An Algebraic Solution of the GPS Equations*. IEEE Trans. Aerosp. Electron. Syst.
- Strang, Borre (1997). *Linear Algebra, Geodesy, and GPS*. Wellesley-Cambridge.
- Hofmann-Wellenhof et al. (2001). *GPS: Theory and Practice*. Springer.

8.9 Exercises

Exercise 8.1. For 4 satellites at $(20000, 0, 20000)$, $(0, 20000, 20000)$, $(-20000, 0, 20000)$, $(0, -20000, 20000)$ with $\tau_i = 25000$, compute the user position and clock bias. Are the satellites on a quadric with focus at the solution?

Exercise 8.2. Show that if all satellites lie on a sphere centered at the user position, the GPS equations have exactly two solutions.

Exercise 8.3. Implement the rank-3 quadratic solver for the ambiguous case. Test on a coplanar satellite configuration.

Exercise 8.4 (Research)**.** Analyze the effect of ionospheric delay (frequency-dependent) on the Minkowski geometry formulation. How does dual-frequency GPS resolve this?

CHAPTER 9

3D Printing of Invariant Manifolds in Dynamical Systems

Patrick R. Bishop, Summer Chenoweth, Emmanuel Fleurantin, Alonso Ogueda-Oliva, Evelyn Sander, Julia Seay
Communicated by *Notices* Associate Editor Vidit Nanda

"Invariant manifolds play a crucial role in understanding the behavior of dynamical systems... Visualizing and intuitively grasping their complex geometries remains challenging. This article introduces a novel approach: 3D printing tangible, physical representations of invariant manifolds."

— Bishop et al.

Abstract. Stable and unstable manifolds organize the long-term behavior of dynamical systems. They are typically visualized as 2D projections of 3D objects—losing crucial geometric information. This chapter presents a complete pipeline: compute local manifolds via the Parameterization Method, extend globally via numerical integration, mesh and thicken for 3D printing, and produce physical models that reveal intersections, spirals, and folding invisible on screens.

9.1 Introduction: Why Print Manifolds?

Consider the Lorenz system:

$$\dot{x} = \sigma(y - x), \quad \dot{y} = x(\rho - z) - y, \quad \dot{z} = xy - \beta z.$$

The origin is a saddle with a 2D stable manifold and 1D unstable manifold. On screen, these appear as overlapping curves. In your hand, the stable manifold is a *twisted sheet* that folds back on itself; the unstable manifold is a *spiral ribbon*. The physical object reveals the *heteroclinic tangle*—the engine of chaos.

This chapter gives you the complete computational pipeline to:

1. Compute local invariant manifolds via the **Parameterization Method** (high-order Taylor approximation).
2. Extend to **global manifolds** via adaptive arclength integration.
3. Generate **watertight meshes** with proper thickening.
4. Prepare for **FDM/SLA printing** with supports and orientation.

9.2 Background: Invariant Manifolds

Definition 9.1 (Stable/Unstable Manifolds)**.** *For $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x})$ with hyperbolic equilibrium $\boldsymbol{x}^*$, let $\boldsymbol{A} = D\boldsymbol{f}(\boldsymbol{x}^*)$ have eigenvalues $\lambda_1, \ldots, \lambda_n$ with $\mathrm{Re}(\lambda_i) \neq 0$.*

- $W^s(\boldsymbol{x}^*) = \{\boldsymbol{x} : \varphi_t(\boldsymbol{x}) \to \boldsymbol{x}^* \text{ as } t \to +\infty\}$ *(stable manifold)*
- $W^u(\boldsymbol{x}^*) = \{\boldsymbol{x} : \varphi_t(\boldsymbol{x}) \to \boldsymbol{x}^* \text{ as } t \to -\infty\}$ *(unstable manifold)*

Dimensions: $\dim W^s = \#\{\mathrm{Re}(\lambda) < 0\}$, $\dim W^u = \#\{\mathrm{Re}(\lambda) > 0\}$.

Theorem 9.2 (Stable Manifold Theorem)**.** *W^s and W^u are smooth immersed submanifolds tangent to the stable/unstable eigenspaces of $\boldsymbol{A}$ at $\boldsymbol{x}^*$.*

9.3 Phase 1: Local Manifolds via the Parameterization Method

The Parameterization Method (Cabré, Fontich, de la Llave) finds a map $P : U \subset \mathbb{R}^k \to \mathbb{R}^n$ such that:

1. $P(0) = \boldsymbol{x}^*$ (equilibrium at origin of parameter domain)
2. $DP(0)$ spans the target eigenspace
3. Invariance: $DP(\boldsymbol{\theta})\boldsymbol{\Lambda}\boldsymbol{\theta} = \boldsymbol{f}(P(\boldsymbol{\theta}))$ for $\boldsymbol{\theta} \in U$

where $\boldsymbol{\Lambda} = \mathrm{diag}(\lambda_1, \ldots, \lambda_k)$ are the target eigenvalues.

Expanding P as a power series $P(\boldsymbol{\theta}) = \sum_{|\boldsymbol{\alpha}| \geq 1} P_{\boldsymbol{\alpha}} \boldsymbol{\theta}^{\boldsymbol{\alpha}}$ and matching coefficients gives linear equations for $P_{\boldsymbol{\alpha}}$:

$$(\boldsymbol{\Lambda}\boldsymbol{\alpha} \cdot \boldsymbol{I} - \boldsymbol{A}) P_{\boldsymbol{\alpha}} = \boldsymbol{R}_{\boldsymbol{\alpha}},$$

where $\boldsymbol{R}_{\boldsymbol{\alpha}}$ depends on previously computed coefficients. Non-resonance condition: $\boldsymbol{\Lambda}\boldsymbol{\alpha} \neq \lambda_j$ for all j.

9.4 Phase 2: Global Extension

The local parameterization P is accurate only near $\boldsymbol{x}^*$ (radius $\sim$ convergence of series). To reach global structure:

1. Sample the **boundary** of the local manifold: $\partial D = \{P(\boldsymbol{\theta}) : \|\boldsymbol{\theta}\| = R\}$.
2. For each boundary point, integrate $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x})$ forward (for W^s) or backward (for W^u) using adaptive Runge-Kutta.

Algorithm 3 Parameterization Method (local stable manifold)

```
Compute equilibrium x*, Jacobian A = Df(x*)
Select k stable eigenvalues λ_1, ..., λ_k with eigenvectors v_1, ..., v_k
Set P_{e_i} = v_i for i = 1, ..., k (linear terms)
for m = 2 to N (order) do
    for all multi-indices α with |α| = m do
        Compute R_α from known P_β (|β| < m)
        Solve (Σ α_i λ_i I − A) P_α = R_α
    end for
end for
Return polynomial P(θ)
```

3. Track the **arclength** to ensure uniform sampling.

4. Use **shadowing** or **manifold continuation** (e.g., `MATCONT`, `COCO`) for bifurcations.

Listing 9.1: Global manifold extension

```julia
using DifferentialEquations, LinearAlgebra

function extend_manifold(P, theta_boundary, f, tspan; dt=0.01)
    # theta_boundary: matrix of boundary parameter values (k x M)
    # Returns: vector of trajectory points on global manifold
    M = size(theta_boundary, 2)
    trajectories = Vector{Vector{Vector{Float64}}}(undef, M)

    for i = 1:M
        u0 = P(theta_boundary[:, i])
        prob = ODEProblem(f, u0, tspan)
        sol = solve(prob, Tsit5(), saveat=dt, abstol=1e-9, reltol=1e-9)
        trajectories[i] = [sol.u[j] for j in 1:length(sol.u)]
    end
    return trajectories
end
```

9.5 Phase 3: Meshing and Thickening

Raw trajectories are 1D (unstable) or 2D point clouds (stable). For 3D printing, we need a **watertight closed mesh**.

1. **Delaunay triangulation** of point cloud (for 2D manifolds).

2. **Poisson surface reconstruction** (Kazhdan et al.) for noisy/unstructured data.

3. **Thickening**: Offset the surface along normal by $\pm\epsilon/2$ to create a solid of thickness ϵ (typically 1–2 mm for FDM).

4. **Manifold repair**: Remove non-manifold edges, fill holes, ensure consistent normals.

Listing 9.2: Mesh generation and thickening

```julia
using Meshing, GeometryBasics, FileIO

function points_to_mesh(points; thickness=1.5)
    # points: N x 3 matrix of points on the manifold
    # Returns: watertight mesh (vertices, faces)

    # Poisson surface reconstruction (requires normal estimates)
    normals = estimate_normals(points, k=20)
    mesh = poisson_reconstruct(points, normals; depth=8)

    # Thicken: create offset surfaces
    inner = offset_mesh(mesh, -thickness/2)
    outer = offset_mesh(mesh, +thickness/2)

    # Combine into closed solid
    solid = union_meshes(inner, outer)
    repair_mesh!(solid)
    return solid
end

function estimate_normals(points, k)
    # PCA on k-nearest neighbors
    tree = KDTree(points')
    normals = zeros(size(points))
    for i in 1:size(points, 1)
        idxs, _ = knn(tree, points[i, :], k+1)
        idxs = idxs[2:end]  # exclude self
        cov = cov(points[idxs, :]')
        _, eigvecs = eigen(cov)
        normals[i, :] = eigvecs[:, 1]  # smallest eigenvalue direction
    end
    return normals
end
```

9.6 Phase 4: Print Preparation

- **Orientation**: Print with the flattest face down; minimize overhangs $> 45°$.
- **Supports**: Tree supports for delicate spirals; grid supports for flat sheets.
- **Material**: PLA (easy), PETG (tougher), Resin (SLA, highest detail).
- **Scale**: Lorenz stable manifold fits in $10 \times 10 \times 10$ cm at scale 1:1.
- **Infill**: 15-20% gyroid for structural stability with minimal weight.

Example 9.3 (Lorenz system parameters)**.** $\sigma = 10$, $\rho = 28$, $\beta = 8/3$. *Equilibrium at origin: eigenvalues* $\approx -22.8, 11.8, -1.8$.

- W^s*: 2D, spanned by* λ_1, λ_3*. Compute order-10 parameterization.*
- W^u*: 1D, integrate backward from local unstable eigenvector.*

- *Result: Two spiraling ribbons intersecting in a heteroclinic tangle.*

9.7 Code Repository

Complete pipeline (MATLAB/Julia/Python) at:

https://github.com/msander/invariant-manifold-3d-print

Includes:

- `ParameterizationMethod.m` — high-order Taylor coefficients
- `GlobalExtension.jl` — adaptive integration
- `MeshAndThicken.py` — Poisson reconstruction + offset
- `STLExport` — print-ready files for Lorenz, Arneodo-Coullet-Tresser, Langford systems

9.8 Bridge to Chapter 10

We've seen how to *visualize* and *fabricate* geometric objects from dynamical systems. Chapter 10 addresses a different challenge: how to *compose* complex systems from simpler components using the mathematics of operads—category theory for system design.

9.9 Further Reading

- Bishop et al. (2026). *3D Printing of Invariant Manifolds*. Notices, 73(1).
- Cabré, Fontich, de la Llave (2003). *The Parameterization Method*. J. Diff. Eq.
- Krauskopf, Osinga (2005). *Growing 1D and Quasi-2D Unstable Manifolds*. Nonlinearity.
- MATLAB package `MATCONT`, Julia package `BifurcationKit.jl`.

9.10 Exercises

Exercise 9.1. Implement the Parameterization Method for the 2D stable manifold of the Lorenz origin at order 5. Compare the Taylor polynomial error vs. distance from origin.

Exercise 9.2. For the Arneodo-Coullet-Tresser system ($\dot{x} = y, \dot{y} = z, \dot{z} = -ax - by - cz + dx^3$), compute and print the unstable manifold of the origin.

Exercise 9.3. Design a 3D-printed model showing the *homoclinic tangle* of the Smale horseshoe. How do you represent the infinite folding in a finite print?

Part IV

Modern Applications

CHAPTER 10

Exciting Coding Problems for DNA-based Storage Systems

Daniella Bar-Lev, Omer Sabary, Eitan Yaakobi
Communicated by *Notices* Associate Editor Richard Levine

> "DNA storage generates new exciting and challenging theoretical problems to our community. The success of such solutions is crucial due to the never-ending growth of data and the foreseeable inability to store it."
>
> — Bar-Lev, Sabary, Yaakobi

Abstract.DNA storage offers ultra-dense, durable data archiving but introduces a unique error model: insertions, deletions, and substitutions (edit errors), unordered strands with massive redundancy, and constrained sequences (GC-content, homopolymers). This chapter surveys the coding theory challenges: edit-distance codes, clustering algorithms, sequence reconstruction, and constrained codes.

10.1 Introduction: Why DNA Storage?

DNA offers remarkable storage properties:

- **Density**: $\sim 10^{18}$ bytes/mm^3 (exabyte per cubic millimeter)
- **Durability**: Thousands of years if kept cool/dry
- **Energy**: No power to maintain; only to read/write
- **Timelessness**: DNA sequencing technology will never become obsolete

But the **channel model** is unlike any classical storage:

Errors Insertions, deletions, substitutions (edit errors), not just bit flips

Ordering Strands are *unordered* (no address track)

Redundancy Millions of copies per strand (unique to DNA)

Constraints GC-content (45-55%), no homopolymers > 3

10.2 Edit-Distance Codes

Classical codes (RS, LDPC, BCH) correct *substitutions/erasures*. Edit errors shift the entire subsequent sequence.

Definition 10.1 (Levenshtein Distance)**.** *$d_L(x, y)$ = minimum number of insertions, deletions, substitutions to transform x into y.*

Theorem 10.2 (Levenshtein 1965)**.** *A code corrects k deletions iff it corrects any combination of k insertions/deletions.*

Example 10.3 (Varshamov-Tenengolts (VT) Codes)**.** *For $a \in \{0, \ldots, n\}$, the length-n VT code is*

$$VT_a(n) = \left\{ x \in \{0,1\}^n : \sum_{i=1}^{n} i x_i \equiv a \pmod{n+1} \right\}.$$

$VT_0(n)$ is a perfect single-deletion-correcting code*. For any a, $|VT_a(n)| \approx 2^n/(n+1)$.*

Theorem 10.4 (VT Code Optimality)**.** *It is open whether $VT_0(n)$ is* strictly optimal *for all n. Conjectured yes (Sloane, 2002).*

10.3 The DNA Clustering Problem

Each synthesized strand has millions of copies. After sequencing, we get an unordered multiset of noisy reads. Before decoding, we must *cluster* reads by their original strand.

Problem 10.5 (DNA Clustering)**.** *Given N reads of length L, partition into clusters $C_1, \ldots, C_M$ such that reads in C_j originate from the same synthesized strand.*

Approaches:

1. **Edit-distance clustering**: Hierarchical clustering with d_L (slow, $O(N^2)$)
2. **Index-based**: Encode strand ID in the sequence (reduces capacity)
3. **Locality-sensitive hashing**: Approximate edit distance for scalability

Current methods fail at high error rates ($> 5\%$) and large N ($> 10^6$).

10.4 Sequence Reconstruction from Noisy Copies

After clustering, each cluster has k noisy copies of one strand. Goal: reconstruct the original.

Problem 10.6 (Trace Reconstruction)**.** *Given k independent traces of $x \in \{0,1\}^n$ through a deletion channel (each bit deleted with prob. p), reconstruct x.*

Theorem 10.7 (Batu et al. 2004)**.** *For deletion probability p, $k = \exp(O(\log^{1/3} n))$ traces suffice for reconstruction with high probability.*

DNA adds *substitutions and insertions*. The minimum cluster size for unique reconstruction is unknown for edit channels.

10.5 Constrained Codes

Synthesis/sequencing errors depend on sequence content. Two key constraints:

Run-length No more than 3 identical consecutive bases (e.g., AAAA forbidden)

GC-content Fraction of G/C in $[0.45, 0.55]$

Theorem 10.8 (Capacity of Constrained Codes)**.** *For a constrained system $S \subset \{A, C, G, T\}^*$, the capacity is*

$$C = \lim_{n \to \infty} \frac{\log_2 |S \cap \{A, C, G, T\}^n|}{n}.$$

For run-length ≤ 3, $C \approx 1.94$ bits/symbol (vs 2 unconstrained). GC-balanced adds another small penalty.

Open problem: Efficient encoding/decoding for *joint* run-length + GC constraints.

10.6 Code: VT Code Encoding/Decoding

Listing 10.1: VT code single-deletion correction

```python
def vt_encode(message_bits, n):
    """Encode message into VT_0(n) codeword."""
    # Systematic encoding: find redundancy bits to satisfy sum(i*x_i) \
       equiv 0 (mod n+1)
    # This is a sketch; real implementation uses systematic encoding
       algorithms
    pass

def vt_decode(received, n):
    """Decode received word with at most one deletion."""
    L = len(received)
    if L == n:
        # No deletion, check syndrome
        syndrome = sum((i+1)*int(b) for i,b in enumerate(received)) % (n
           +1)
        if syndrome == 0:
            return received  # Valid codeword
        # Single substitution error - correctable
    elif L == n-1:
        # One deletion - find position
        syndrome = sum((i+1)*int(b) for i,b in enumerate(received)) % (n
           +1)
        # Syndrome gives position of deleted bit
        # Insert 0 or 1 at that position to satisfy VT constraint
    return None

# Test
n = 10
codeword = vt_encode([1,0,1,1], n)
print("Codeword:", codeword)
# Simulate deletion
```

```
import random
del_pos = random.randint(0, n-1)
received = codeword[:del_pos] + codeword[del_pos+1:]
decoded = vt_decode(received, n)
print("Decoded:", decoded)
```

10.7 Bridge to Chapter 11

DNA storage coding theory solves *channel-specific* problems. Chapter 11 addresses *system-level* design: how to compose complex systems from simpler components using operads—a categorical framework for syntax and semantics of system composition.

10.8 Further Reading

- Bar-Lev, Sabary, Yaakobi (2022). *Exciting Coding Problems for DNA-based Storage*. Notices, 69(11).
- Sloane (2002). *On Single-Deletion-Correcting Codes*. IEEE Trans. Inf. Theory.
- Church et al. (2012). *Next-Generation Digital Information Storage in DNA*. Science.
- Carmean et al. (2019). *DNA Data Storage and Hybrid Molecular–Electronic Computing*. IEEE.

10.9 Exercises

Exercise 10.1. Implement the VT syndrome decoder for $n = 15$. Test on all possible single-deletion errors for all codewords in $VT_0(15)$.

Exercise 10.2. Design a run-length limited code (no AAAA, CCCC, GGGG, TTTT) with rate ≥ 1.9 bits/symbol. Implement encoder/decoder.

Exercise 10.3. Simulate the DNA clustering problem: generate 1000 strands with 100 copies each, add 3% edit errors, and cluster using edit distance + hierarchical clustering. What fraction of clusters are pure?

Exercise 10.4 (Research)**.** Investigate polar codes for edit-distance channels. Can the polarization phenomenon be adapted to insertion/deletion channels?

CHAPTER 11

The Mathematics of Cyber Defense

John A. Emanuello, Ahmad Ridley
Communicated by *Notices* Associate Editor Emilie Purvine

> "The speed, complexity, and ubiquity of cyber-attacks has never been more apparent... Current cyber-defense capabilities are static and rules-based... This approach is unsustainable."
>
> — Emanuello & Ridley

Abstract. Cyber defense requires detecting anomalies in massive, heterogeneous data streams (logs, netflow, alerts) and responding autonomously. This chapter surveys the mathematical foundations: autoencoders for unsupervised anomaly detection, Word2Vec-style embeddings for categorical log data, and reinforcement learning for autonomous response. The unique challenges: adversarial dynamics, non-stationary distributions, and the need for human-machine teaming.

11.1 Introduction: Why Mathematics for Cyber?

Cyber attacks are not single events but *campaigns*: multi-phase, adaptive, stealthy. Defenders face:

- **Volume**: Terabytes/day of logs, netflow, packet captures
- **Heterogeneity**: Categorical (IPs, usernames), temporal, graph-structured
- **Adversariality**: Attackers adapt to defenses
- **Label scarcity**: Almost no labeled attacks; must detect *anomalies*

Mathematical approach: **Build models of normal behavior; flag deviations.**

11.2 Data Representation: Embedding Categorical Features

Log data is nominal: IP addresses, usernames, process names, file paths. Standard ML needs numeric vectors.

Word2Vec for logs (Mikolov et al., 2013): Treat a log line as a "sentence," tokens as "words." Train skip-gram model:

$$\max_{\theta} \sum_{(w,c)\in D} \log P(c|w;\theta), \quad P(c|w) = \frac{\exp(\boldsymbol{v}_c^\top \boldsymbol{v}_w)}{\sum_{c'} \exp(\boldsymbol{v}_{c'}^\top \boldsymbol{v}_w)}.$$

Result: Each token (IP, process, etc.) gets a dense vector $\boldsymbol{v} \in \mathbb{R}^d$. Similar behavior $\rightarrow$ nearby vectors.

Listing 11.1: Log embedding with Word2Vec

```
from gensim.models import Word2Vec
import numpy as np

# Parse logs into token sequences
log_sequences = [
    ["192.168.1.1", "ssh", "accept", "user1"],
    ["192.168.1.2", "ssh", "failed", "user2"],
    # ...
]

# Train Word2Vec
model = Word2Vec(log_sequences, vector_size=100, window=5, min_count=1,
    workers=4)

# Embed a log line
def embed_log(tokens, model):
    vecs = [model.wv[t] for t in tokens if t in model.wv]
    return np.mean(vecs, axis=0) if vecs else np.zeros(model.vector_size)

# Now each log is a 100-d vector for downstream ML
```

11.3 Anomaly Detection: Deep Autoencoders

Deep Autoencoder (AE): Multi-layer perceptron trained to reconstruct input:

$$\boldsymbol{z} = \sigma(\boldsymbol{W}_e \boldsymbol{x} + \boldsymbol{b}_e), \quad \hat{\boldsymbol{x}} = \sigma(\boldsymbol{W}_d \boldsymbol{z} + \boldsymbol{b}_d),$$

loss $= \|\boldsymbol{x} - \hat{\boldsymbol{x}}\|^2$. The bottleneck $\boldsymbol{z}$ learns a compressed representation.

Unsupervised anomaly detection: Train on *normal* traffic only. At test time, high reconstruction error $\rightarrow$ anomaly.

Theorem 11.1 (AE as Nonlinear PCA)**.** *A deep AE with linear activations learns the PCA subspace. With nonlinearities, it learns a* nonlinear manifold *of normal data.*

Listing 11.2: Autoencoder for log anomaly detection

```
import torch
import torch.nn as nn

class LogAutoencoder(nn.Module):
    def __init__(self, input_dim, latent_dim=32):
```

```
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 128), nn.ReLU(),
            nn.Linear(128, 64), nn.ReLU(),
            nn.Linear(64, latent_dim)
        )
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 64), nn.ReLU(),
            nn.Linear(64, 128), nn.ReLU(),
            nn.Linear(128, input_dim)
        )

    def forward(self, x):
        z = self.encoder(x)
        return self.decoder(z), z

# Training on normal data only
model = LogAutoencoder(input_dim=100)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(50):
    for batch in normal_dataloader:
        recon, _ = model(batch)
        loss = nn.MSELoss()(recon, batch)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

# Anomaly score = reconstruction error
def anomaly_score(model, x):
    with torch.no_grad():
        recon, _ = model(x)
        return torch.mean((x - recon)**2, dim=1)
```

11.4 Chaining Anomalies Across Modalities

A single anomaly (e.g., unusual process) may be benign. A *chain* across modalities (unusual process + unusual netflow + unusual file access) is likely an attack.

Challenge: Fuse AE scores from log embeddings, netflow embeddings, alert embeddings into a unified timeline.

Approach: **Temporal point processes** or **graph neural networks** on the alert correlation graph.

11.5 Autonomous Response: Reinforcement Learning

Detection is not enough; must *respond*. Formulate as **Markov Decision Process**:

- State s_t: Network state, active alerts, system health
- Action a_t: Block IP, isolate host, rate-limit, collect forensics

- Reward r_t: Negative for damage, positive for containment, cost for disruption

Learn policy $\pi(a|s)$ via **Deep Q-Network (DQN)** or **Proximal Policy Optimization (PPO)**.

Listing 11.3: RL environment for cyber response

```python
import gymnasium as gym
import numpy as np

class CyberEnv(gym.Env):
    def __init__(self):
        # State: [num_alerts, severity_scores, network_health, ...]
        self.observation_space = gym.spaces.Box(-np.inf, np.inf, (20,))
        # Actions: 0=monitor, 1=block_ip, 2=isolate_host, 3=rate_limit, 4=
            forensics
        self.action_space = gym.spaces.Discrete(5)
        self.state = None
        self.attack_ongoing = False

    def step(self, action):
        # Simulate attack progression and response effect
        reward = self._compute_reward(action)
        terminated = self.attack_neutralized()
        truncated = self.steps >= 100
        return self.state, reward, terminated, truncated, {}

    def reset(self, seed=None):
        self.state = np.zeros(20)
        self.attack_ongoing = np.random.rand() < 0.3
        return self.state, {}
```

Human-in-the-loop: RL agent proposes actions; human analyst approves/modifies. Over time, trust increases → more autonomy.

11.6 Bridge to Chapter 12

Cyber defense uses deep learning for *pattern recognition* in dynamic systems. Chapter 12 uses **operads** (categorical algebra) for *compositional design* of complex systems. Both address the challenge of *structure in complexity*: one learns it, the other specifies it.

11.7 Further Reading

- Emanuello, Ridley (2022). *The Mathematics of Cyber Defense*. Notices, 69(6).
- Golczynski, Emanuello (2021). *End-to-End Anomaly Detection....* arXiv:2108.12276.
- Sutton, Barto (2018). *Reinforcement Learning*. MIT Press.
- Goodfellow et al. (2016). *Deep Learning*. MIT Press (Ch. 14 on AEs).
- MITRE ATT&CK framework: https://attack.mitre.org

11.8 Exercises

Exercise 11.1. Train a Word2Vec model on the HDFS log dataset. Visualize the embedding of IP addresses—do malicious IPs cluster?

Exercise 11.2. Implement a variational autoencoder (VAE) for log anomalies. Compare reconstruction error vs. ELBO as anomaly scores.

Exercise 11.3. Design a reward function for a cyber RL agent that balances: (1) minimizing data exfiltration, (2) minimizing false positive isolations, (3) minimizing analyst workload.

Exercise 11.4 (Research)**.** Investigate *adversarial attacks on the anomaly detector itself*: can an attacker craft traffic that looks normal to the AE but is malicious?

CHAPTER 12

Operads for Designing Systems of Systems

John C. Baez, John D. Foley
Communicated by *Notices* Associate Editor Emilie Purvine

> "Analogous to how a group abstracts the symmetries of an object, an operad abstracts operations that compose many objects into a single object."
>
> — Baez & Foley

Abstract."Systems of systems" engineering composes independent systems (radar, comms, weapons, sensors) into a larger capability. This chapter introduces **operads**—a categorical tool that formalizes the *syntax* of composition (how things plug together) and their *algebras* (the concrete semantics). Applied to network design, search-and-rescue planning, and task coordination.

12.1 Introduction: The Composition Problem

A smartphone is a system of systems: touchscreen, camera, CPU, radio, GPS, battery... The *design* is a blueprint for how components connect. The *behavior* depends on the specific hardware.

In defense (DARPA CASCADE program), we need to compose:

- Aircraft with range-limited comms
- Ships, helicopters, drones for search-and-rescue
- Sensors, effectors, decision-makers for task planning

Challenge: The composition rules (syntax) are different from the physical realization (semantics). Operads separate these.

12.2 What Is an Operad?

Definition 12.1 (Colored Operad)**.** *An **operad** $\mathcal{O}$ consists of:*

- *A set of **types** (colors)* $\mathrm{Ob}(\mathcal{O})$
- *For each tuple $(X_1, \ldots, X_n; Y)$ with $X_i, Y \in \mathrm{Ob}(\mathcal{O})$, a set of **operations** $\mathcal{O}(X_1, \ldots, X_n; Y)$*
- ***Composition**: Given $f \in \mathcal{O}(X_1, \ldots, X_n; Y)$ and $g_i \in \mathcal{O}(Z_{i1}, \ldots, Z_{ik_i}; X_i)$,*
$$f \circ (g_1, \ldots, g_n) \in \mathcal{O}(Z_{11}, \ldots, Z_{nk_n}; Y)$$
- ***Identities**:* $\mathrm{id}_X \in \mathcal{O}(X; X)$
- ***Symmetric group actions**: Permuting inputs*

Satisfying associativity, identity, equivariance axioms.

Definition 12.2 (Operad Algebra)**.** *An **algebra** of $\mathcal{O}$ assigns to each type X a set $A(X)$ and to each operation $f \in \mathcal{O}(X_1, \ldots, X_n; Y)$ a function*
$$A(f) : A(X_1) \times \cdots \times A(X_n) \to A(Y)$$
preserving composition, identities, and symmetries.

Intuition: Operad = *syntax* (how to compose); Algebra = *semantics* (what the components actually are).

12.3 Network Operads

Example 12.3 (Aircraft Communication Network)**.** ***Types**: Number of aircraft $n \in \mathbb{N}$.*

***Operations**: $\mathcal{O}(n_1, \ldots, n_k; m)$ = graphs with $n_1 + \cdots + n_k$ input vertices and m output vertices. An operation tells you* which new edges to add *to connect the sub-networks.*

***Algebra**: For each n, $A(n)$ = set of possible states of n aircraft (positions, velocities, comms status). An operation acts by adding comms links between aircraft that are in range.*

Theorem 12.4 (Network Operad Construction)**.** *For any class of networks (graphs, directed graphs, typed edges, etc.), there is a network operad whose operations are "ways to connect networks by adding edges."*

12.4 Application: Search and Rescue

Problem: Distribute search assets (ships, planes, helicopters, small boats) to cover a large area after a disaster (Fastnet Race 1979, Sydney-Hobart 1998).

1. **Design operad**: Assets have "slots" (helicopter carries 3 boats). Operations = ways to nest assets.
2. **Algebra**: Actual assets at actual locations with fuel constraints.
3. **Optimization**: Search over operad operations to maximize "search effort" (area covered per time) within budget.

The operad structure lets you *search the space of designs* algorithmically.

12.5 From Design to Tasking

Same operad, different algebra:

- **Design algebra**: Assets at locations → network structure
- **Tasking algebra**: Primitive tasks ("fly to", "search", "rescue") composed into mission plans

Catalyst operads (Baez et al.): Some operations enable other operations (a "catalyst" agent coordinates multiple agents). Used for multi-agent task plans.

Translation to **constraint programming** (MILP/CPLEX): Each operation → decision variables; composition → constraints.

12.6 Levels of Abstraction

A system design proceeds through levels:

1. **Abstract**: "Network of boats with comms"
2. **Concrete**: "3 RHIBs, 2 helicopters, SATCOM links"
3. **Detailed**: Specific models, frequencies, protocols

Operad homomorphisms and **algebra homomorphisms** formalize moving between levels. This is *change of abstraction*.

12.7 Code: Simple Operad in Python

Listing 12.1: Network operad sketch

```
from dataclasses import dataclass
from typing import List, Dict, Set, Callable
from abc import ABC, abstractmethod

@dataclass
class NetworkOperation:
    """Operation: connect input networks by adding edges"""
    input_types: List[int]  # sizes of input networks
    output_type: int        # size of output network
    edges: Set[tuple]       # (source_idx, target_idx) where idx is global

    def __call__(self, *networks):
        # Compose networks according to edges
        # networks[i] has size input_types[i]
        pass

class Operad:
    def __init__(self):
        self.operations: Dict[tuple, List[NetworkOperation]] = {}

    def add(self, op: NetworkOperation):
        key = (tuple(op.input_types), op.output_type)
```

```
        self.operations.setdefault(key, []).append(op)

    def compose(self, outer: NetworkOperation, inners: List[
        NetworkOperation]):
        # Build composed operation
        # Offset inner edge indices by input network sizes
        pass

class Algebra:
    def __init__(self, interpret_type: Callable[[int], set]):
        self.interpret_type = interpret_type  # n -> set of n-aircraft
            states

    def apply(self, op: NetworkOperation, *states):
        # Apply operation to concrete states
        pass

# Example: Aircraft comms
network_operad = Operad()
# Add operations for connecting networks...

# Algebra: real aircraft positions and comms ranges
def aircraft_states(n):
    # Return set of possible states for n aircraft
    # Each state: positions, velocities, comms equipment
    pass

aircraft_algebra = Algebra(aircraft_states)
```

12.8 Bridge to Chapter 13

Operads provide a *static* compositional structure for system design. Chapter 13 applies *dynamic* Bayesian inference to a different domain: astronomy. Both deal with *hierarchical structure*—operads explicitly, astronomy through the hierarchy of data → model → cosmology.

12.9 Further Reading

- Baez, Foley (2021). *Operads for Designing Systems of Systems*. Notices, 68(6).
- Baez, Foley, Moeller, Pollard (2020). *Network Models*. Theory Appl. Categ.
- Spivak (2013). *The Operad of Wiring Diagrams*. arXiv:1305.0297.
- Spivak (2014). *Category Theory for the Sciences*. MIT Press.
- Yau (2018). *Operads of Wiring Diagrams*. Springer.

12.10 Exercises

Exercise 12.1. Define the operad of *wiring diagrams* for digital circuits. Types = natural numbers (wire count). Operations = circuits with given input/output wires.

Exercise 12.2. For the aircraft comms operad, define an algebra where $A(n)$ is the set of possible communication graphs on n aircraft with given range constraints.

Exercise 12.3. Design a "catalyst" operation for a search-and-rescue scenario where a helicopter coordinates the deployment of 3 small boats.

Exercise 12.4 (Research)**.** Extend the network operad to include *temporal* composition: operations that describe not just spatial connections but time-dependent interactions (e.g., intermittent connectivity).

Part V

Data-Intensive Science

CHAPTER 13

A Brief History of Inference in Astronomy

Rafael S. de Souza, Emille E. O. Ishida, Alberto Krone-Martins
Communicated by *Notices* Associate Editor Richard A. Levine

> "Throughout history, methods of inference have been deeply linked to advances in astronomy."
>
> — de Souza, Ishida, Krone-Martins

Abstract. From Hipparchus averaging solstice times to Hamiltonian Monte Carlo sampling cosmological parameters, astronomy has driven the development of statistical inference. This chapter traces the arc: least squares (Gauss/Ceres), Bayesian revival (Jeffreys), MCMC (cosmology), to modern likelihood-free and amortized inference. The lesson: astronomical data challenges force statistical innovation.

13.1 Introduction: Astronomy as an Inference Engine

Astronomy is fundamentally an **inverse problem**: we observe photons (or gravitational waves, neutrinos) and infer the physical universe. Every major advance in data → model inference has an astronomical milestone:

Hipparchus (190–120 BC)	Averaging → noise reduction
Tycho Brahe (1546–1601)	Repeated observations → precision
Gauss (1801)	Least squares → orbit of Ceres
Laplace (1812)	Central Limit Theorem → error theory
Jeffreys (1939)	Bayesian theory → cosmology
MCMC (1990s–)	Cosmological parameter estimation
HMC/Nested Sampling (2000s–)	High-dimensional posteriors
Likelihood-free (2010s–)	Simulator-based inference
Amortized (2020s–)	Neural posterior estimation

13.2 Linear Regression and Least Squares

Gauss (1801) predicted the orbit of Ceres using **least squares**. The problem: fit an ellipse to noisy angular observations.

Model: $\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\theta} + \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \boldsymbol{\Sigma})$.
MLE/least squares: $\hat{\boldsymbol{\theta}} = (\boldsymbol{X}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{y}$.
This is still the workhorse for:

- Hubble diagram (distance vs. redshift)
- Black hole mass scaling relations
- Spectral line fitting

13.3 Bayesian Inference in Cosmology

Type Ia supernovae (1998) revealed cosmic acceleration. The inference problem:

- Data: $\{\mu_i, z_i\}$ = distance modulus + redshift
- Model: $\mu(z; \Omega_m, \Omega_\Lambda, H_0)$ from Friedmann eqs
- Posterior: $p(\boldsymbol{\theta}|\mathcal{D}) \propto p(\mathcal{D}|\boldsymbol{\theta}) p(\boldsymbol{\theta})$

Computational challenge: 6–10 parameters, multimodal, degenerate. MCMC (Metropolis-Hastings) became standard.

13.4 Modern Sampling: HMC and Nested Sampling

Hamiltonian Monte Carlo Uses gradient of log-posterior to propose distant states with high acceptance. Essential for ~ 100-parameter models (e.g., Planck CMB).

Nested Sampling Computes evidence $Z = \int p(\mathcal{D}|\boldsymbol{\theta}) p(\boldsymbol{\theta}) d\boldsymbol{\theta}$ for model comparison (e.g., ΛCDM vs. wCDM).

Listing 13.1: HMC for cosmological parameters

```python
import numpy as np
import jax.numpy as jnp
from jax import grad, jit
import numpyro
import numpyro.distributions as dist
from numpyro.infer import MCMC, NUTS

def cosmological_model(obs_mu, obs_z, obs_sigma):
    # Flat priors
    Omega_m = numpyro.sample('Omega_m', dist.Uniform(0.0, 1.0))
    Omega_L = numpyro.sample('Omega_L', dist.Uniform(0.0, 1.0))
    H0 = numpyro.sample('H0', dist.Uniform(50.0, 80.0))

    # Distance modulus from cosmology
    def dist_modulus(z):
```

```
        # Simplified; real version integrates Friedmann eq
        return 5*jnp.log10(luminosity_distance(z, Omega_m, Omega_L, H0)) +
            25

    mu_pred = jnp.array([dist_modulus(z) for z in obs_z])

    # Likelihood
    numpyro.sample('obs', dist.Normal(mu_pred, obs_sigma), obs=obs_mu)

# Run NUTS (HMC)
kernel = NUTS(cosmological_model)
mcmc = MCMC(kernel, num_warmup=1000, num_samples=2000)
mcmc.run(jax.random.PRNGKey(0), obs_mu, obs_z, obs_sigma)
samples = mcmc.get_samples()
```

13.5 Likelihood-Free Inference (LFI)

Many astronomical models have *intractable likelihoods* but are easy to *simulate*:

- Galaxy formation (hydrodynamics + gravity)
- Gravitational waveforms (numerical relativity)
- Strong lensing (ray tracing)

Simulation-based inference: Learn $p(\boldsymbol{\theta}|\mathcal{D})$ without evaluating $p(\mathcal{D}|\boldsymbol{\theta})$.

Approximate Bayesian Computation (ABC) Accept $\boldsymbol{\theta}$ if $d(\mathcal{D}_{\text{sim}}, \mathcal{D}_{\text{obs}}) < \epsilon$

Neural Density Estimation Train normalizing flow on $(\boldsymbol{\theta}, \mathcal{D}_{\text{sim}})$ pairs

Score-based Learn score function $\nabla_{\mathcal{D}} \log p(\mathcal{D}|\boldsymbol{\theta})$

13.6 Amortized Inference

Amortization: Instead of running MCMC for each new dataset, train a neural network once to output the posterior:

$$\phi^* = \arg\min_{\phi} \mathbb{E}_{\boldsymbol{\theta},\mathcal{D}\sim p(\boldsymbol{\theta})p(\mathcal{D}|\boldsymbol{\theta})} \left[D_{\text{KL}}(p(\boldsymbol{\theta}|\mathcal{D}) \| q_\phi(\boldsymbol{\theta}|\mathcal{D})) \right].$$

Examples: `swyft`, `sbi`, `zuko` packages. Critical for LSST (10M alerts/night) and gravitational wave real-time analysis.

13.7 Bridge to Chapter 14

Astronomy drives *methodological* innovation. Chapter 14 shows how *engineering* principles (FAIR) preserve the software that implements these methods. Both are about *longevity*: astronomical methods last centuries; FAIR ensures code lasts decades.

13.8 Further Reading

- de Souza, Ishida, Krone-Martins (2025). *A Brief History of Inference in Astronomy*. Notices, 72(10).
- Stigler (1986). *The History of Statistics*. Harvard.
- Trotta (2017). *Bayesian Methods in Cosmology*. arXiv:1701.01467.
- Alsing et al. (2019). *Simulation-Based Inference*. arXiv:1903.00007.
- Cranmer et al. (2020). *The Frontier of Simulation-Based Inference*. PNAS.

13.9 Exercises

Exercise 13.1. Implement least-squares orbit fitting for a synthetic asteroid with 6 observations. Compare to Gauss's original method.

Exercise 13.2. Run MCMC on the Union2.1 supernova dataset (580 SNe Ia) to constrain Ω_m, Ω_Λ. Plot the confidence contours.

Exercise 13.3. Train a normalizing flow (RealNVP) to approximate the posterior of a 2D cosmological model. Compare to MCMC samples.

Exercise 13.4 (Research)**.** Design an amortized inference pipeline for real-time gravitational wave parameter estimation (latency < 1 second).

CHAPTER 14

Making Mathematical Online Resources FAIR

Tabea Bacher, Marina Garrote-López, Christiane Görgen, Marius J. Neubert
Communicated by *Notices* Associate Editor

> "Digital preservation today is guided by the FAIR Principles—Findability, Accessibility, Interoperability, and Reusability. These principles do not insist on the newest and shiniest web tool... much more sensibly, they guide our decisions on selecting methodologies that preserve the long-term value and usability of digitized results."
>
> — Bacher et al.

Abstract. Many valuable mathematical databases (OEIS, LMFDB, Small Phylogenetic Trees) were built with early-2000s technology. This chapter presents a case study: modernizing the *Small Phylogenetic Trees* library using FAIR principles. We separate computation from presentation, use OSCAR/Julia for verified computation, and deploy a versioned, documented, licensed resource at `algebraicphylogenetics.org`.

14.1 Introduction: The Legacy Software Crisis

Mathematical software ages differently than papers. A 2005 paper is still readable; a 2005 Maple worksheet may not run. The *Small Phylogenetic Trees* library (Garcia, Puente, 2005) computed phylogenetic invariants for trees up to 5 taxa using Singular and Maple. By 2025:

- Original URLs broken (institution changes)
- Maple/Singular versions incompatible
- No DOI, no license, no API

- Data in GIF images and unstructured text files

FAIR principles (Wilkinson et al., 2016) provide a roadmap:

F Findable: Persistent identifiers (DOI), rich metadata, indexed

A Accessible: Standard protocols (HTTPS, API), authentication if needed

I Interoperable: Standard formats (JSON, MathML, HDF5), controlled vocabularies

R Reusable: Clear license, provenance, community standards

14.2 Case Study: Small Phylogenetic Trees

Original resource: Tables of algebraic invariants (phylogenetic ideals) for group-based models (Jukes-Cantor, Kimura 2/3-parameter) on trees with 3, 4, 5 leaves.

Mathematical content:

- Tree topology T (graph with labeled leaves)
- Nucleotide substitution model $\to$ polynomial map $\phi_T : \mathbb{C}^p \to \mathbb{C}^{4^n}$
- Phylogenetic ideal $I_T = \ker \phi_T$ (polynomials vanishing on the model)
- Invariants: generators of I_T, dimension, degree, ML degree

Modernization pipeline:

1. **Computation**: Recompute all invariants in `OSCAR` (Julia), a computer algebra system with formal verification.
2. **Serialization**: Store as `.mrdi` (MaRDI) files—JSON-based, schema-validated, with full provenance (code version, input, timestamp).
3. **Website**: `algebraicphylogenetics.org` built with `Flask` + `MathJax`, serving data from PostgreSQL with REST API.
4. **Versioning**: Git repository for code; Zenodo for data snapshots with DOIs.
5. **License**: CC-BY-4.0 for data, MIT for code.

14.3 The MaRDI Format

The **MaRDI** (Mathematical Research Data Initiative) format structures computational results:

Listing 14.1: Example .mrdi file for a phylogenetic invariant

```
{
  "@context": "https://mardi4nfdi.github.io/metadata/schema.json",
  "@type": "MathematicalComputation",
  "identifier": "doi:10.5281/zenodo.1234567",
  "name": "Phylogenetic invariant for 4-leaf Jukes-Cantor model",
  "description": "Generator of the phylogenetic ideal I_T...",
  "hasInput": {
```

```
    "@type": "MathematicalObject",
    "name": "4-leaf tree topology",
    "representation": {"format": "newick", "value": "((A,B),(C,D));"}
  },
  "hasOutput": {
    "@type": "MathematicalObject",
    "name": "Phylogenetic invariant",
    "representation": {"format": "polynomial", "value": "p0011*p0100 -
       p0001*p0110 + ..."}
  },
  "usedSoftware": {
    "@type": "Software",
    "name": "OSCAR",
    "version": "0.12.0",
    "url": "https://github.com/oscar-system/OSCAR.jl"
  },
  "dateCreated": "2025-03-15T14:32:00Z",
  "license": "CC-BY-4.0"
}
```

14.4 Verification and Reproducibility

Key innovation: The new computation is *verified*:

- OSCAR uses exact rational arithmetic (no floating-point errors)
- Gröbner basis computations are certified
- Results cross-checked with `Macaulay2` and `Singular`

The `.mrdi` file contains the *entire provenance*: software versions, input parameters, random seeds. Anyone can re-run and bitwise verify.

Listing 14.2: Verified computation in OSCAR

```
using Oscar
using Pkg; Pkg.activate("AlgebraicPhylogenetics")

# Exact computation of phylogenetic ideal for 4-leaf JC69
T = phylogenetic_tree("((A,B),(C,D));")
model = jukes_cantor_model()
I = phylogenetic_ideal(T, model)

# Verify generators match known results
gens = minimal_generators(I)
println("Number of generators: ", length(gens))
println("Degrees: ", [total_degree(g) for g in gens])

# Export to MaRDI
export_mardi(I, "invariant_4leaf_jc69.mrdi")
```

14.5 Guidelines for FAIRifying Your Resource

1. **Audit**: What data/code exists? What formats? What dependencies?
2. **Separate concerns**: Computation $\neq$ presentation. Build a *software package* (versioned, tested) + a *website* (consumes package output).
3. **Choose standards**:
   - Data: HDF5, NetCDF, Parquet, or MaRDI JSON
   - Metadata: Schema.org, CodeMeta, MaRDI
   - API: OpenAPI/Swagger
4. **Assign DOIs**: Zenodo, Figshare, or institutional repository
5. **License explicitly**: CC-BY-4.0 (data), MIT/BSD (code)
6. **Document provenance**: Every output links to code version, inputs, environment
7. **Test reproducibility**: CI pipeline that re-runs computations on every commit

14.6 Bridge to Chapter 15

FAIR principles ensure mathematical software survives institutional changes. Chapter 15 applies similar rigor to *tomographic reconstruction*: a complete pipeline from raw projections to readable text, with documented code, versioned data, and reproducibility—exactly the FAIR principles in action for inverse problems.

14.7 Further Reading

- Bacher et al. (2026). *Making Mathematical Online Resources FAIR*. Notices, 73(5).
- Wilkinson et al. (2016). *The FAIR Guiding Principles*. Scientific Data.
- MaRDI Consortium (2024). *MaRDI White Paper*. https://mardi4nfdi.de
- Garcia-Puente et al. (2005). *Small Phylogenetic Trees*. https://algebraicphylogenetics.org
- OSCAR Computer Algebra System. https://www.oscar-system.org

14.8 Exercises

Exercise 14.1. Take a small mathematical dataset (e.g., a table of integer sequences) and create a MaRDI JSON file for it. Include input, output, software version, and license.

Exercise 14.2. Set up a GitHub Actions workflow that re-runs a Julia/OSCAR computation on every push and verifies the output matches a stored .mrdi file.

Exercise 14.3 (Research)**.** Design a FAIR-compliant schema for storing trained neural network models (weights, architecture, training data provenance, hyperparameters) that enables reproducibility and comparison.

CHAPTER 15

Deciphering Scrolls with Tomography: A Training Experiment

Sonia Foschiatti, Axel Kittenberger, Otmar Scherzer
Communicated by *Notices* Associate Editor Krystal Taylor

> "Virtual unwrapping uses software to reveal text that was hidden for centuries, thereby making it accessible to mankind."
>
> — Foschiatti, Kittenberger, Scherzer

Abstract.X-ray computed tomography (CT) enables non-destructive reading of damaged scrolls. This chapter presents a complete didactic pipeline: from simulated X-ray projections (using visible light and transparent films) through filtered back-projection, manual segmentation, and Maximum Intensity Projection (MIP) to readable text. Includes Arduino-controlled rotation stage, Python/MATLAB reconstruction code, and the Archeolab software framework.

15.1 Introduction: The Virtual Unwrapping Problem

Damaged scrolls (Herculaneum papyri, En-Gedi scroll, medieval manuscripts) cannot be physically opened. X-ray CT provides a solution:

1. **Scan**: Rotate scroll, collect projections (radiographs)
2. **Reconstruct**: Filtered back-projection $\rightarrow$ 3D volume
3. **Segment**: Identify individual layers in the volume
4. **Texture/Map**: Assign intensity values to surface points
5. **Flatten**: Map 3D surface $\rightarrow$ 2D image
6. **Merge**: Combine segments into complete text

This chapter implements the full pipeline as an **educational laboratory** using visible light instead of X-rays.

15.2 The Mathematical Model: Radon Transform

Let $f(x, y)$ be the attenuation coefficient in a slice. A parallel-beam projection at angle θ:

$$p_\theta(s) = \int_{-\infty}^{\infty} f(s\cos\theta - t\sin\theta, s\sin\theta + t\cos\theta)dt.$$

This is the **Radon transform** $\mathcal{R}f(\theta, s)$.

Inversion (filtered back-projection):

$$f(x, y) = \int_0^{\pi} [p_\theta * h]\,(x\cos\theta + y\sin\theta)d\theta,$$

where h is the **Ram-Lak filter** (ramp filter in Fourier domain): $\hat{h}(\omega) = |\omega|$.

15.3 Experimental Setup: Visible-Light Tomography

Hardware:

- Arduino-controlled stepper motor (rotation stage)
- Projector (visible light source) + Fresnel lens (collimation)
- Camera (detector) + white screen
- Transparent cellulose acetate sheets with printed text (phantom scroll)

Acquisition:

Listing 15.1: Arduino stepper control + camera sync

```
# Arduino sketch (simplified)
#include <Stepper.h>
const int stepsPerRevolution = 200;
Stepper myStepper(stepsPerRevolution, 8, 9, 10, 11);
int nProjections = 800;

void setup() {
    myStepper.setSpeed(10);  // rpm
    Serial.begin(9600);
}

void loop() {
    if (Serial.available()) {
        char cmd = Serial.read();
        if (cmd == 's') {  // start acquisition
            for (int i = 0; i < nProjections; i++) {
                myStepper.step(stepsPerRevolution / nProjections);
                delay(500);  // settle
                Serial.println("CAPTURE");  // trigger camera
                delay(500);
            }
        }
    }
}
```

15.4 Reconstruction Pipeline

15.4.1 1. Preprocessing

- Dark/flat field correction
- Rotation axis determination (cross-correlation of 0° and 180° projections)
- Cropping to region of interest

15.4.2 2. Filtered Back-Projection (FBP)

Listing 15.2: FBP reconstruction using iradon

```
import numpy as np
from skimage.transform import iradon

# projections: (n_angles, n_pixels) array
# angles: in degrees
reconstruction = iradon(projections, theta=angles,
                        filter='ramp',  # Ram-Lak
                        interpolation='linear',
                        circle=False)

# Post-process: circular mask to remove edge artifacts
from skimage.draw import disk
mask = np.zeros_like(reconstruction)
rr, cc = disk((n/2, n/2), n/2 - 5)
mask[rr, cc] = 1
reconstruction *= mask
```

15.4.3 3. Segmentation: Finding Layers

Each slice is a cross-section through the scroll. The layers appear as concentric curved lines.

Listing 15.3: Spiral segmentation via genetic algorithm

```
import numpy as np
from scipy.optimize import differential_evolution

def spiral_model(theta, params):
    # params = [center_x, center_y, a, b]
    cx, cy, a, b = params
    r = a + b * theta
    return cx + r * np.cos(theta), cy + r * np.sin(theta)

def segmentation_loss(params, slice_img):
    # Generate spiral mask from params
    # Compare to image gradients
    # Return negative correlation
    pass

# Optimize per slice
result = differential_evolution(segmentation_loss, bounds,
```

```
                                    args=(slice_img,), maxiter=100)
best_spiral = result.x
```

15.4.4 4. Texture Mapping and Flattening

For each layer, extract intensity values along the spiral, map to a rectangular image.

Listing 15.4: Layer flattening

```
def flatten_layer(volume, spiral_params, layer_thickness=3):
    """Extract and flatten one layer from 3D volume."""
    h, w, d = volume.shape
    flattened = np.zeros((d, n_points))

    for z in range(d):
        slice_img = volume[:, :, z]
        cx, cy, a, b = spiral_params[z]

        # Sample along spiral
        theta = np.linspace(0, 4*np.pi, n_points)
        x = cx + (a + b*theta) * np.cos(theta)
        y = cy + (a + b*theta) * np.sin(theta)

        # Interpolate
        from scipy.ndimage import map_coordinates
        flattened[z, :] = map_coordinates(slice_img, [y, x], order=1)

    return flattened
```

15.4.5 5. Maximum Intensity Projection (MIP)

Combine multiple layers into one readable image:

$$\text{MIP}(u, v) = \max_z I_z(u, v).$$

Invert if text is dark-on-light.

15.5 Results: Reconstructing the Herculaneum Scrolls

The article demonstrates reconstruction of:

- **Greek**: Homer, *Hymni Homerici XV* (Hercules hymn)
- **Latin**: Lucretius, *De Rerum Natura* (Book 1, lines 72–74)

15.6 Code Repository: Archeolab

Complete pipeline (MATLAB + Python) at:

https://gitlab.com/csc1/archeolab/

Includes:

- `guiAxisFinder.m` — rotation axis detection
- `guiSegment.m` — manual + genetic algorithm segmentation
- `FBP_reconstruction.m` — filtered back-projection
- `MIP_flattening.m` — texture mapping + flattening
- Jupyter notebooks for Python equivalents
- Pre-acquired datasets for testing without hardware

15.7 Bridge to Chapter 14

Tomography reconstructs *hidden structure* from projections. Chapter 14 shows how *FAIR principles* reconstruct *usable knowledge* from legacy mathematical software. Both are inverse problems: from fragmented observations to coherent understanding.

15.8 Further Reading

- Foschiatti, Kittenberger, Scherzer (2025). *Deciphering Scrolls with Tomography.* Notices, 72(10).
- Seales et al. (2016). *From Damage to Discovery via Virtual Unwrapping.* Science Advances.
- Natterer (2001). *The Mathematics of Computerized Tomography.* SIAM.
- Feeman (2015). *The Mathematics of Medical Imaging.* Springer.
- Vesuvius Challenge: https://vesuviuschallenge.org

15.9 Exercises

Exercise 15.1. Implement the Radon transform and its adjoint (back-projection) from scratch in NumPy. Verify $\mathcal{R}^*\mathcal{R}$ is a convolution with $1/|x|$.

Exercise 15.2. Build the visible-light tomography setup (projector + camera + Arduino stage). Acquire 180 projections of a printed spiral phantom and reconstruct with FBP.

Exercise 15.3. The Ram-Lak filter amplifies high-frequency noise. Implement a *windowed* ramp filter (Hann, Hamming, Shepp-Logan) and compare reconstruction quality vs. noise amplification.

Exercise 15.4 (Research)**.** The Herculaneum scrolls have carbon-based ink (low X-ray contrast). Develop a phase-contrast CT simulation and reconstruction pipeline. How much does phase retrieval improve ink visibility?

Part VI

The Future

Bibliography